

时间序列分析 课程论文

题目：基于时间序列模型的比特币与以太坊价格关系研究

班级：信计 2302 班

专业：信息与计算科学

学号：1043220122

姓名：林鑫

日期：2025 年 11 月

基于时间序列模型的比特币与以太坊价格关系研究

摘要

本报告以 2020 年 1 月至 2025 年 10 月比特币（BTC）与以太坊（ETH）的每日收盘价数据为研究对象，系统地构建并分析了其价格及收益率序列的统计特征与动态关系。实证结果显示，两者的原始价格序列均非平稳，但对数收益率序列平稳，且呈现出显著的“尖峰厚尾”和“波动聚集”特征。在一元分析中，本文采用 ARIMA(1, 0, 1) 模型有效刻画了收益率的线性趋势，并进一步引入考虑 t-分布的 GARCH(1, 1) 模型成功捕捉了市场的条件异方差性和极端风险特征。基于该联合模型，实现了对 BTC 未来价格走势及 95% 置信区间的有效预测。在多元分析部分，Johansen 协整检验表明 BTC 与 ETH 之间不存在长期稳定的协整关系，不具备建立 VECM 模型的基础。然而，格兰杰因果检验揭示了两者在短期内存在显著的双向信息溢出效应。进一步构建的 VAR(10) 模型及其脉冲响应分析（IRF）和方差分解（FEVD）结果表明，加密市场存在显著的不对称传导机制：BTC 具有强主导性，其价格冲击能迅速正向传导至 ETH；而 ETH 则表现出明显的跟随特征，其自身波动对 BTC 的反向影响微弱。本研究不仅验证了 ARIMA-GARCH-t 模型在高波动加密资产风险计量中的适用性，也量化揭示了 BTC 作为市场领导者与 ETH 作为跟随者的动态联动结构，为数字资产投资组合管理提供了重要的实证依据。

关键词：比特币；以太坊；ARIMA-GARCH-t；VAR 模型；格兰杰因果检验；波动溢出

目录

基于时间序列模型的比特币与以太坊价格关系研究	1
摘要	2
1. 引言	4
2. 数据基本统计分析	4
2.1 数据来源	4
2.2 数据基本统计分析数据	5
3. 一元时间序列建模	13
3.1 基本原理	13
3.2 ARIMA-GARCH 建模	14
3.3 最终模型表达式	18
3.4 模型预测	19
3.5 风险价值 (VaR) 回测分析 (基于 GARCH(1,1)-t 模型)	20
4. 多元时间序列	22
4.1 协整性检验	23
4.2 格兰杰因果检验 (Granger Causality Test)	23
4.3 VAR (向量自回归) 模型	24
4.4 脉冲响应分析 (IRF)	27
4.5 方差分解分析 (Variance Decomposition, FEVD)	30
5. 总结与展望	32
5.1 主要研究结论	32
5.2 投资启示与建议	33
5.3 研究局限性与未来展望	33
附录: python 代码	34

1. 引言

随着数字经济的蓬勃发展，加密货币市场已成为全球金融体系中不可忽视的新兴板块。作为市值最高、影响力最大的两种核心资产，比特币（BTC）和以太坊（ETH）的价格行为不仅反映了市场情绪的变迁，也蕴含了复杂的风险传递机制。然而，加密市场典型的高波动性、“尖峰厚尾”分布以及资产间复杂的动态关联，使得传统的线性金融模型在分析和预测时面临严峻挑战。为了更准确地刻画和理解这两种资产的价格动态，本研究采用了多层次的时间序列分析框架。首先，针对单一资产收益率普遍存在的波动聚集和非正态分布特征，传统的 GARCH 模型往往难以充分捕捉极端风险。因此，本研究引入了基于 t-分布的 GARCH 模型（GARCH-t），旨在更精确地描述收益率的尾部特征和时变波动率。其次，在探究资产间关系时，区分长期均衡与短期联动至关重要。本研究通过 Johansen 协整检验考察了两者是否存在长期稳定的价格锚定关系，并通过 VAR 模型框架下的格兰杰因果检验、脉冲响应函数及方差分解技术，深入解构了两者在短期内的信息传导方向和强度，试图回答“谁在引领市场”这一核心问题。本报告旨在通过严谨的实证分析，揭示 BTC 与 ETH 市场的内在运行规律，验证成熟金融计量模型在新兴加密市场中的有效性，并为投资者进行风险规避和策略制定提供量化参考。

2. 数据基本统计分析

2.1 数据来源

本实验数据均来源于 CoinGecko 网站下载，截取了 2020-01-01 至 2025-10-30 的每日收盘价单位为 USDT，以下为数据预览：

date	btc_close	eth_close
2020-01-01 00:00:00 UTC	7195.153895430029	129.18638529253852

date	btc_close	eth_close
2020-01-02 00:00:00 UTC	7193.7546679601	130.4846850311141
2020-01-03 00:00:00 UTC	6963.853353983485	127.04525801179804
2020-01-04 00:00:00 UTC	7300.34693530511	133.70264861844188
2020-01-05 00:00:00 UTC	7351.377053166627	134.1368826978575

2.2 数据基本统计分析数据

2.2.1 基本数据特征

下表展示了 `btc\_close` (比特币收盘价) 和 `eth\_close` (以太坊收盘价) 的核心描述性统计数据。

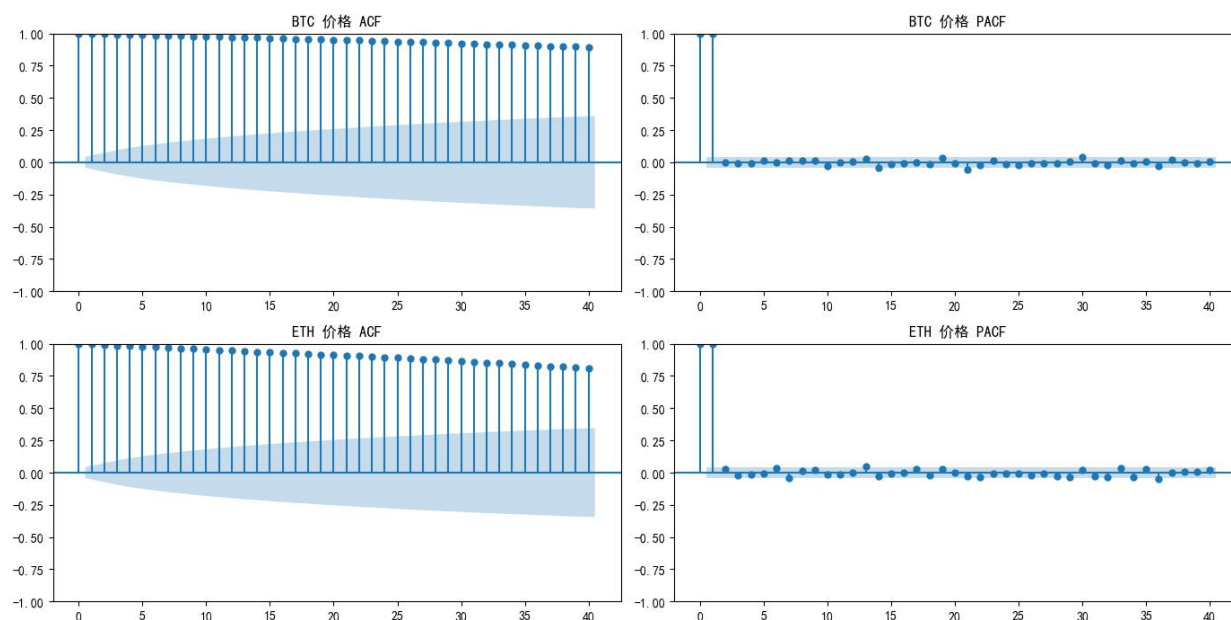
统计量	btc_close	eth_close
计数 (count)	2130.000000	2130.000000
均值 (mean)	45832.318269	2134.436787
标准差 (std)	30839.402336	1183.327106
最小值 (min)	5032.501351	110.597898
25%分位数 (25%)	21428.063943	1384.076317
中位数 (50%)	38646.722846	2002.150320
75%分位数 (75%)	62868.922659	3056.962393
最大值 (max)	124773.508231	4829.225542

2.2.2 数据可视化

图1：原始价格时序图 (BTC vs ETH)



图4：原始价格的自相关性（非平稳性展示）



原始价格时序图及其自相关性分析

该图表（图 1）展示了从 2020 年初到 2025 年中期的比特币（`btc_close`）和以太坊（`eth_close`）的每日收盘价时间序列。通过观察图 1 及其自相关图（图 4），我们可以总结出以下几个显著的时序特征：

1. 显著的长期趋势（Trend）

两个时间序列（BTC 和 ETH）均表现出明显的长期上升趋势。尽管过程中伴随着剧烈的价格回调，但从 2020 年初的低点到 2025 年的高点，整体的价格中枢是持续向上移动的。

2. 周期性波动（Cyclicity）

两个序列都具有非常明显的周期性特征，即牛市和熊市的交替。例如，在 2021 年至 2022 年初附近出现了一个显著的价格高峰，随后进入了一个较长的下跌和盘整期（熊市），直到 2024 年初某个时候开始进入下一个上升周期（牛市），并在 2024 年至 2025 年间创下新的高点。

这种周期性波动是不规律的，而非严格的“季节性”（Seasonality），因为其发生的时间间隔和持续长度并不固定。

3. 波动率聚集性（Volatility Clustering）

这是一个非常关键的金融时序特征。两个序列的波动性（Volatility）不是恒定的。在价格快速上涨或快速下跌的时期（如 2021 年、2024-2025 年），价格的日内或日间摆动幅度显著增大，表现为图形上的“尖峰”和剧烈震荡。而在价格处于低位盘整期时（如 2020 年、2023 年），价格的波动性相对较低。这种“高波动率时期”和“低波动率时期”聚集出现的现象，称为波动率聚集。

4. 非平稳性（Non-Stationarity）

基于以上特征，这两个时间序列都是非平稳序列。

均值非平稳：由于存在明显的长期趋势（见图 1），序列的均值随时间而变化。

方差非平稳：由于存在波动率聚集（见图 1），序列的方差（波动性）也随时间而变化。

自相关性证据 (图 4)：图 4 中的自相关图（ACF）和偏自相关图（PACF）为非平稳性提供了决定性证据。

ACF 缓慢衰减：BTC 和 ETH 的 ACF 图均显示出自相关系数（柱状图）缓慢下降，即使在高阶滞后（如 40 阶）时仍然保持高度正相关。这是非平稳序列（特别是含有单位根，如随机游走）的最典型特征。

PACF 一阶截尾：同时，两者的 PACF 图均显示在滞后 1 阶时有非常强的相关性（接近 1.0），而在 1 阶之后迅速“截尾”（落入置信区间内）。

结论：ACF 缓慢衰减和 PACF 一阶截尾是随机游走过程（Random Walk, I(1)过程）的经典标志，这从统计上证明了原始价格序列是非平稳的。

非平稳性是后续进行时间序列建模（如 ARIMA、GARCH 等）时必须首先处理的问题。通常需要通过差分（Differencing）来使其平稳。

5. 高度相关性（High Correlation）

BTC 和 ETH 的价格走势（图 1）表现出极强的正相关性。无论是长期的趋势方向，还是周期性的拐点（高峰和低谷出现的时间），两者都高度一致。这表明它们受到共同的宏观市场因素驱动。

2.2.3 平稳性与白噪声检验

为了在后续步骤中建立有效的时间序列模型，我们必须使用平稳的数据序列。本节对“原始价格”和“对数收益率”序列进行了 ADF 单位根检验和 Ljung-Box 白噪声检验。

1 原始价格平稳性检验 (ADF)

ADF 检验的原假设（H0）是：序列存在单位根，即序列非平稳。

BTC 价格：

`ADF 统计量`：-0.3899

`p-value`：0.9118 (9.1177e-01)

`结论`：p-value 远大于 0.05，我们无法拒绝原假设。序列非平稳。

ETH 价格：

`ADF 统计量`：-1.8763

`p-value`：0.3433 (3.4331e-01)

`结论`：p-value 大于 0.05，我们无法拒绝原假设。序列非平稳。

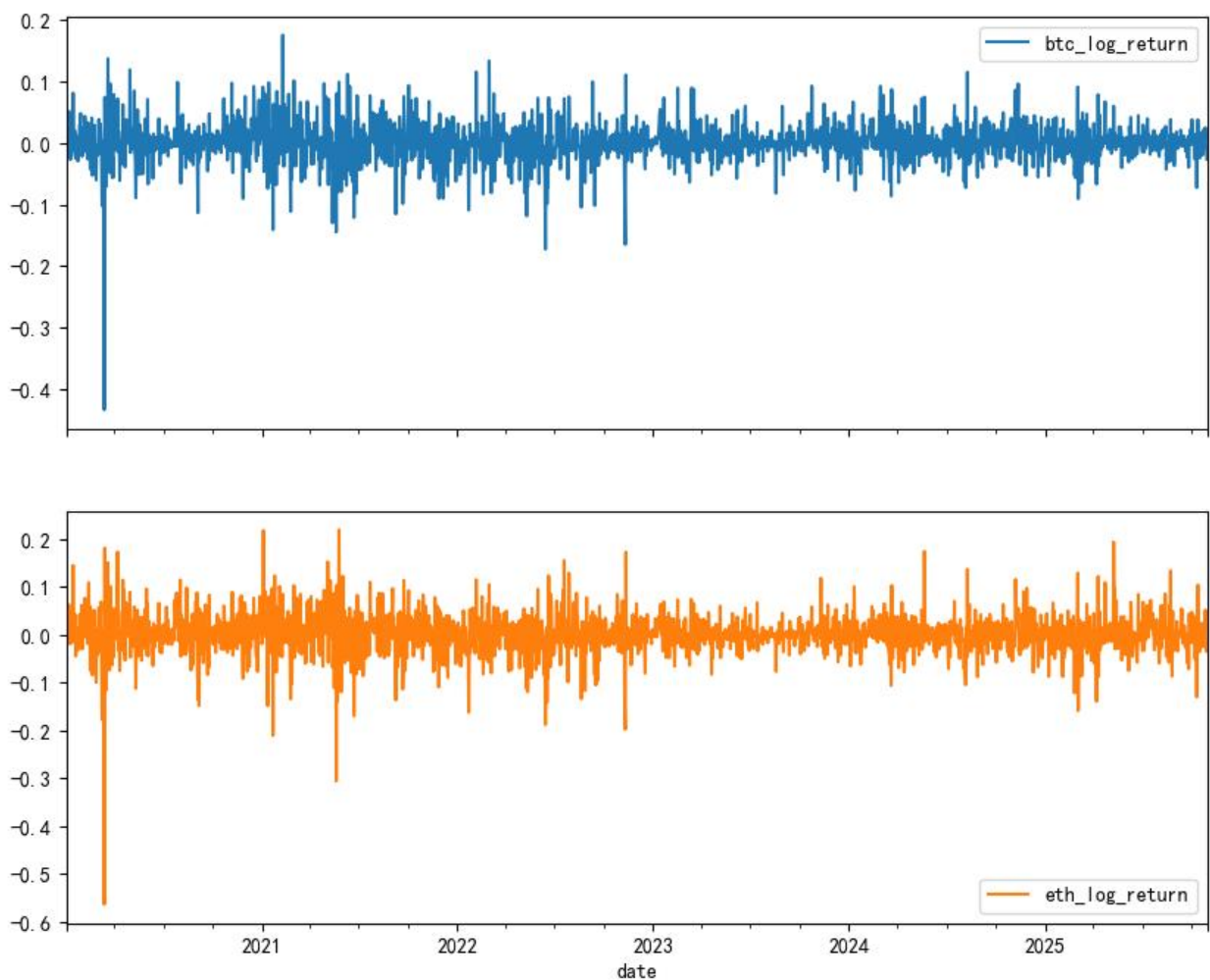
ADF 检验结果在统计上证实了 `图 1` 和 `图 4` 中的视觉观察：BTC 和 ETH 的原始价格序列都是非平稳的，不能直接用于建模。

2.2.4 数据处理

从上面分析我们不难得出两段时间序列均非平稳，我们需要做差分或一些数据处理使其平稳，考虑到金融数据的特征，我们采用对数据取对数后做差分，即对数收益率（ $R_t = \ln(P_t) - \ln(P_{t-1})$ ）

我们对原始价格取对数并进行一阶差分（即“对数收益率”）后，再次进行检验。

图2：对数收益率时序图（BTC vs ETH）



1. 平稳性检验 (ADF)

BTC 对数收益率:

`p-value`: 9.3209e-28

`结论`: p-value 极小 (远小于 0.05), 我们强烈拒绝原假设 (存在单位根)。序列已平稳。

ETH 对数收益率:

`p-value`: 1.8476e-22

`结论`: p-value 极小 (远小于 0.05), 我们强烈拒绝原假设。序列已平稳。

2. 白噪声检验 (Ljung-Box)

在确认序列平稳后, 我们必须检验该序列是否为“白噪声”。白噪声序列是完全随机的, 没有任何自相关性, 因此没有任何可供模型提取的信息。

Ljung-Box 检验的原假设 (H_0) 是: 序列是白噪声 (即不存在自相关)。

BTC 对数收益率:

`p-value`: 0.00677 (6.7725e-03)

`结论`: p-value 小于 0.05, 我们强烈拒绝原假设 (序列为白噪声)。序列非白噪声。

ETH 对数收益率:

`p-value`: 0.000062 (6.2378e-05)

`结论`: p-value 极小 (远小于 0.05), 我们强烈拒绝原假设。序列非白噪声。

3 总体结论

检验结果非常理想:

1. ADF 检验证明, 通过计算对数收益率, 我们已成功将非平稳的原始价格序列转换为了平稳序列。
2. Ljung-Box 检验证明, 这个平稳的收益率序列并不是纯粹的随机白噪声, 它内部仍然存在显著的自相关性 (正如 `图 5` ACF/PACF 图所示)。

这一结果证实了我们的数据是“可建模”的。下一步, 我们可以使用 ARMA、VAR 或 GARCH 等模型来捕捉这些平稳序列中的自相关性。

2.2.5 检验尖峰厚尾与 Q-Q 图

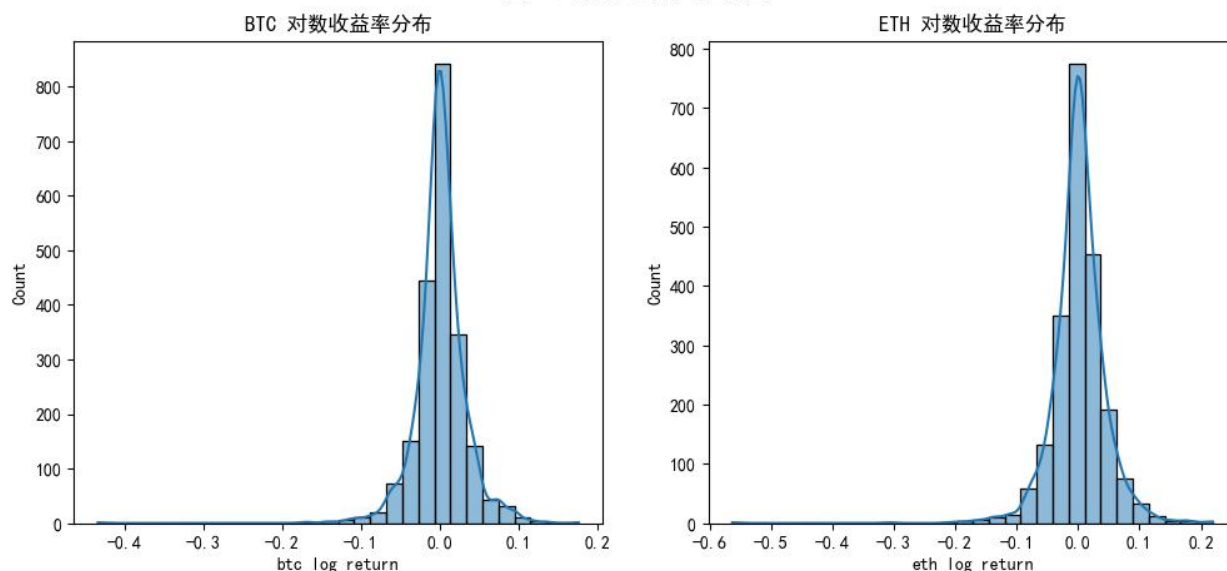
1 收益率分布特征分析 (尖峰厚尾)

在 2.2.4 中, 我们通过取对数和一阶差分, 将非平稳的“价格”序列转换为了平稳的“对数收益率”序列。下一步, 我们必须检验这个收益率序列本身的分布特征, 这对于后续的波动率建模 (如 GARCH) 至关重要。

我们主要分析收益率序列是否符合“正态分布”。

图 3: 对数收益率分布直方图

图3：对数收益率分布直方图



直方图（图 3）展示了 BTC 和 ETH 对数收益率的频数分布。

中心趋势：两个序列的分布都清晰地集中在 0 附近，这符合我们对“收益率”的预期（即平均日收益率接近于 0）。

“尖峰”形态：与标准的“钟形”（正态分布）曲线相比，两个分布在 0 附近都显得异常地高耸和尖锐。这表明，绝大多数的交易日（例如 70-80%）都集中在一个非常窄的微小回报（如 -1% 到 +1%）区间内。

统计检验：偏度 (Skewness) 与 峰度 (Kurtosis)

数字统计为视觉观察提供了更精确的证据：

偏度 (Skewness)：

BTC: -1.1759, ETH: -1.1491

正态分布的偏度为 0。我们的结果是显著的负偏度（“左偏”）。

金融含义：这表明，在数据集中，极端的负收益（暴跌）相比于同等规模的极端正收益（暴涨）更为常见或幅度更大。

峰度 (Kurtosis)：

BTC: 18.0235, ETH: 16.5673

正态分布的（超额）峰度为 0。我们的结果（18.0 和 16.6）是极高的正峰度值。

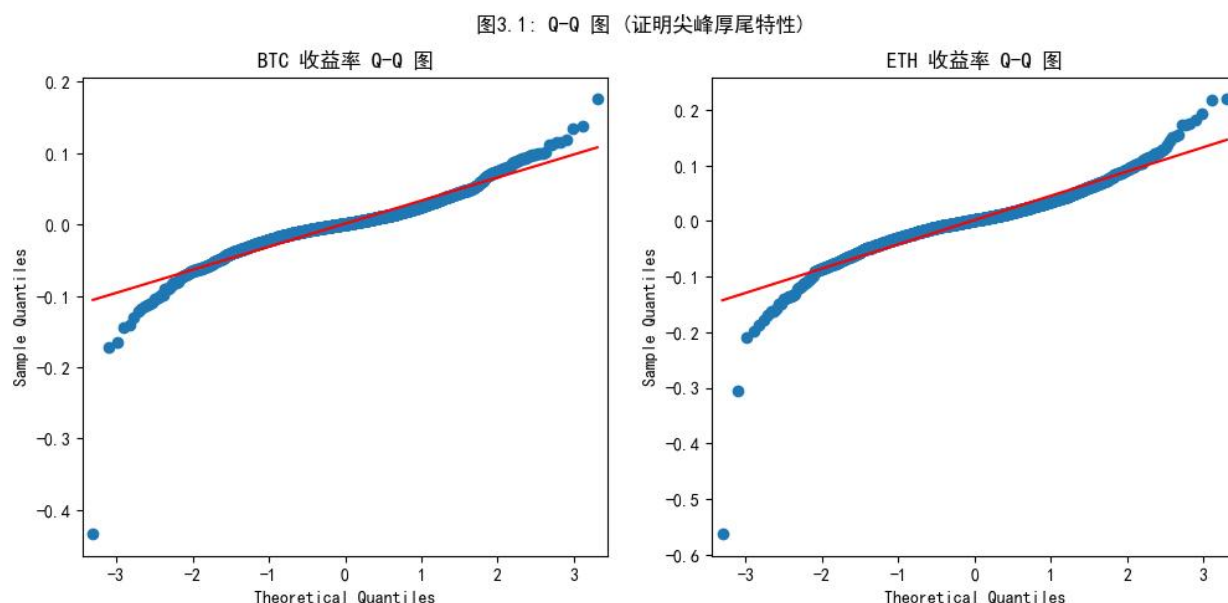
金融含义：这是“尖峰厚尾” (Leptokurtosis / Fat Tails) 的决定性统计证据。

尖峰 (Lepto-)：对应直方图（图 3）中观察到的高耸尖峰。

厚尾 (Fat Tails)：意味着分布的“尾部”（即极端事件）远比正态分布所预测的要

“厚”。换言之，像 $\pm 5\%$ 甚至 $\pm 10\%$ 这样的极端日收益，其发生的概率远高于经典金融模型（假设正态分布）所预期的。

图 3.1: Q-Q 图
Q 图（图 3.1）



是检验“厚尾”特征的最直观工具。它将我们数据的实际分位数（Y 轴）与理论正态分布的分位数（X 轴）进行对比。

如果数据是正态分布：所有数据点都应紧密地落在红色的对角线上。

我们的实际观察：

两个图（BTC 和 ETH）均呈现出清晰的“S”形。

在分布的两个尾端（左下角和右上角），数据点系统性地偏离了红线。

左尾（极端负收益）：数据点位于红线下，表明实际发生的负收益远比正态分布预测的更差。

右尾（极端正收益）：数据点位于红线上方，表明实际发生的正收益远比正态分布预测的更好。

结论

对数收益率序列不服从正态分布。

BTC 和 ETH 的收益率均表现出显著的负偏度和极其强烈的“尖峰厚尾”特性。这一发现（“厚尾”）是金融时间序列最关键的“风格化事实”之一。

对后续建模的指导意义：这个结论意味着，任何假设正态分布的简单模型（如标准 ARIMA）都将严重低估市场极端风险。因此，我们在后续的波动率建模中，必须使用一个能捕捉“厚尾”特性的分布。

3. 一元时间序列建模

3.1 基本原理

1 ARIMA 模型原理

(1) 模型概述

ARIMA (AutoRegressive Integrated Moving Average, 自回归积分滑动平均) 模型是时间序列分析中经典的线性预测模型, 用于描述和预测平稳或经差分后平稳的时间序列。该模型综合了自回归 (AR) 与滑动平均 (MA) 的思想, 并通过差分操作 (I) 将非平稳数据转化为平稳序列。

(2) 模型结构

ARIMA(p, d, q) 模型由三个参数构成:

p: 自回归项阶数, 反映当前值与过去 p 期值之间的线性关系;

d: 差分次数, 用于消除非平稳性趋势;

q: 滑动平均项阶数, 反映当前值与过去 q 期随机扰动的关系。

其一般表达式为:

$$\phi(B)(1 - B)^d y_t = \theta(L)\varepsilon_t$$

其中,

(B) 为滞后算子, ($By_t = y_{t-1}$);

($\phi(B)$)与($\theta(B)$) 分别为自回归与滑动平均多项式;

(ε_t) 为白噪声项。

(3) 建模步骤

ARIMA 模型的典型建模流程包括:

平稳性检验 (ADF 检验等);

差分操作以获得平稳序列;

ACF 与 PACF 分析确定 p、q;

参数估计与显著性检验;

残差白噪声检验;

预测与置信区间分析。

(4) 模型特性

ARIMA 模型能够有效刻画线性依赖关系与趋势性变化, 预测精度较高, 但无法反映波动随时间变化的“条件异方差”特征, 因此常与 GARCH 模型结合使用。

2 GARCH 模型原理

(1) 模型背景

金融市场数据 (尤其是收益率序列) 通常具有显著的波动聚集性: 高波动时期往往伴随高风险, 低波动时期则相对稳定。传统的 ARIMA 模型无法刻画这种方差随时间变化的特征。为此, Engle (1982) 提出 ARCH 模型, Bollerslev (1986) 进一步扩展为 GARCH

（Generalized ARCH）模型，以动态刻画条件方差的演化过程。

（2）模型结构

GARCH(p, q)模型由均值方程与条件方差方程两部分组成：

均值方程：

$$y_t = \mu + \varepsilon_t$$

条件方差方程：

$$\sigma_t^2 = \omega + \sum_{i=1}^p \alpha_i \varepsilon_{t-i}^2 + \sum_{j=1}^q \beta_j \sigma_{t-j}^2$$

其中：

(σ_t^2) ：时刻 t 的条件方差；

(α_i) ：过去冲击对当前波动的影响；

(β_j) ：过去波动对当前波动的持续性影响。

最常用的形式是 GARCH(1,1)：

$$\sigma_t^2 = \omega + \alpha \varepsilon_{t-1}^2 + \beta \sigma_{t-1}^2$$

（3）模型含义

(α) ：代表短期冲击效应（收益突变对波动的即时影响）；

(β) ：代表波动的持续效应；

当 $(\alpha + \beta)$ 接近 1 时，说明波动具有较强的持续性。

3. ARIMA-GARCH 混合模型

在实际的金融时间序列建模中，ARIMA 模型用于刻画序列的均值动态结构，而 GARCH 模型用于刻画残差的方差动态结构。两者结合形成 ARIMA-GARCH 混合模型：

$$\begin{cases} y_t = \mu + \phi(B)(y_{t-1}) + \varepsilon_t \\ \varepsilon_t = \sigma_t z_t, \quad z_t \sim N(0,1) \\ \sigma_t^2 = \omega + \alpha \varepsilon_{t-1}^2 + \beta \sigma_{t-1}^2 \end{cases}$$

该模型能够同时捕捉金融资产价格的线性依赖与波动聚集特征，在数字货币市场（如 BTC、ETH）波动分析与风险建模中具有广泛的应用价值。

3.2 ARIMA-GARCH 建模

第上一节中，我们通过探索性分析和统计检验，识别出了 BTC 对数收益率序列的几个关键特征：

- 1.平稳性 (Stationary): ADF 检验通过 ($p < 0.05$)，序列是平稳的。
- 2.均值自相关性 (Autocorrelation in Mean): Ljung-Box 检验未通过 ($p < 0.05$)，表明序列不是白噪声，其均值中存在线性可预测的模式。
- 3.尖峰厚尾 (Leptokurtosis / Fat Tails): Q-Q 图（图 3.1）呈 "S" 形，且峰度 (Kurtosis) 远大于 3（来自第 2 节），表明极端事件的发生概率远高于正态分布。
- 4.波动率聚集 (Volatility Clustering): ACF 图（图 5）显示收益率的自相关性很小，但从时

序图（图 2）中可以目视观察到高波动和低波动聚集的现象。

我们的建模目标，就是必须同时捕捉到上述所有特征。

标准的线性模型（如普通 OLS 或 ARIMA）只能处理特征 (1) 和 (2)，它们假设波动率是恒定的，这无法解释 (3) 和 (4)。因此，我们采用了 ARIMA-GARCH 的两步建模法。

1 均值方程：ARIMA(1, 0, 1)

目的：捕捉收益率中的线性自相关模式（即特征 2）。

原理：ARIMA(p,d,q) 模型通过自回归（AR）项和移动平均（MA）项来拟合序列的均值。

由于我们的数据已经是平稳的（d=0），因此我们实际上是在使用 ARMA(p,q) 模型。

执行：我们使用 auto_arima_select 函数，通过 AIC 和 BIC 准则，在 (p,q) 从 (0,0) 到 (3,3) 的范围内自动搜索，发现 ARMA(1, 1) 是最优模型。

结果分析：

--- BTC 对数收益率 ARIMA 模型摘要 ---

```
SARIMAX Results
=====
Dep. Variable:          btc_log_return      No. Observations:          2129
Model:                  ARIMA(1, 0, 1)      Log Likelihood              4291.187
Date:                   Tue, 04 Nov 2025    AIC                        -8574.374
Time:                   23:18:31           BIC                        -8551.721
Sample:                 01-02-2020         HQIC                       -8566.083
                   - 10-30-2025
Covariance Type:        opg
=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
const          0.0013      0.001       1.808      0.071      -0.000      0.003
ar.L1         -0.5954      0.185      -3.212      0.001      -0.959     -0.232
ma.L1          0.5436      0.194       2.799      0.005       0.163      0.924
sigma2          0.0010     1.07e-05     97.110      0.000       0.001      0.001
=====
Ljung-Box (L1) (Q):                0.10      Jarque-Bera (JB):          29457.02
Prob(Q):                           0.75      Prob(JB):                  0.00
Heteroskedasticity (H):             0.38      Skew:                      -1.20
Prob(H) (two-sided):               0.00      Kurtosis:                  21.06
=====
```

ARIMA(1,0,1) 模型成功地提取了均值中的自相关性。我们通过以下两点证明了这一点：

BTC 对数收益率 参数显著性 (p-values):

```
const          p = 0.070603 -> 不显著
ar.L1          p = 0.001317 -> 显著
ma.L1          p = 0.005133 -> 显著
sigma2         p = 0.000000 -> 显著
```

BTC 对数收益率 残差 Ljung-Box 检验 (lags = [10, 20]):

```
lag=10 -> lb_stat=14.5256, p=0.150338 -> 通过(白噪声)
lag=20 -> lb_stat=26.2755, p=0.156905 -> 通过(白噪声)
```

系数显著：ar.L1 ($p=0.001$) 和 ma.L1 ($p=0.005$) 均高度显著，常数项 p 值略大于 0.05，认为可以保留，表明该模型结构是有效的。

残差白噪声：最重要的证据是残差 Ljung-Box 检验。在 $\text{lag}=10$ 和 $\text{lag}=20$ 时， p 值分别为 0.150 和 0.157，均大于 0.05。这表明 ARIMA(1, 1) 模型的残差已经是白噪声了，模型成功“滤除”了均值中的所有线性信息。

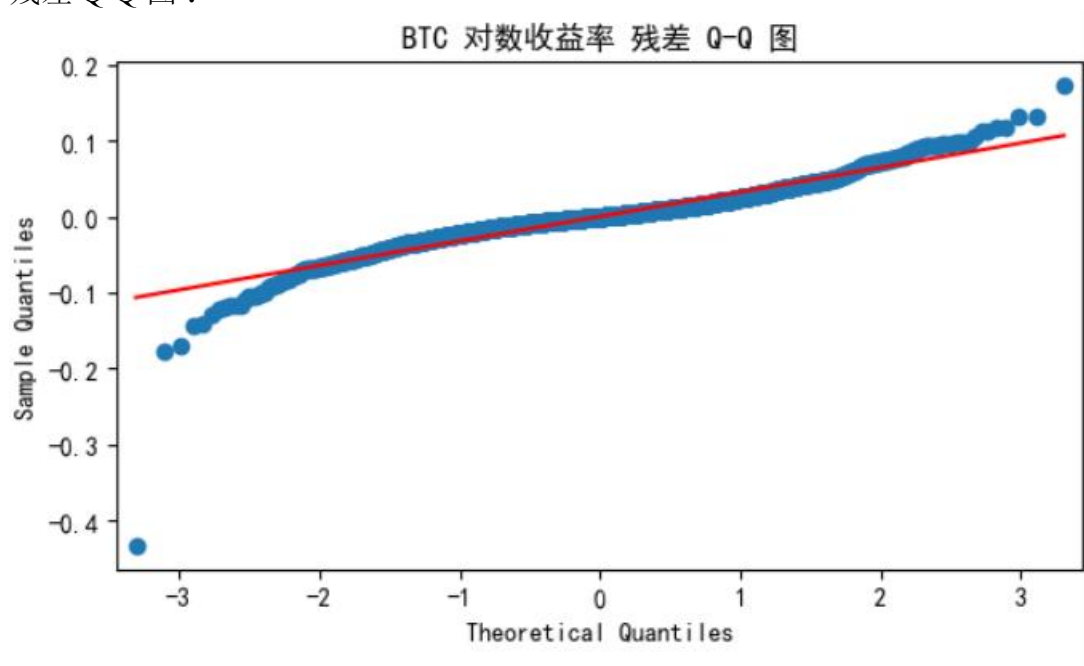
然而，ARIMA 模型是不完整的。它的假设是残差的方差（波动率）是恒定的。

SARIMAX Results 输出强烈反驳了这一点：

Heteroskedasticity (H) 检验：Prob(H) = 0.00 (小于 0.05)。这强烈拒绝了“波动率恒定”的原假设，证明了异方差（即波动率聚集）的存在。

Jarque-Bera (JB) 检验：Prob(JB) = 0.00 (小于 0.05)。这证明了残差不服从正态分布。

残差 Q-Q 图：



该图显示出与原始数据（图 3.1）几乎完全相同的“S”形，证明 ARIMA 完全没有解决“尖峰厚尾”问题。

小结：ARIMA 成功地拟合了“均值”，但其“残差”中仍然充满了“波动率聚集”和“尖峰厚尾”的信息。

2 波动率方程：GARCH(1, 1) 与 t-分布

目的：捕捉 ARIMA 模型残差中的异方差性（特征 4）和尖峰厚尾（特征 3）。

原理：GARCH (广义自回归条件异方差) 模型的核心思想是波动率不是固定的。它建立了一个模型来预测“明天的波动率”，其预测值取决于“今天的波动率”和“今天的冲击（即 ARIMA 残差）”。

GARCH-t：我们已经从 ARIMA 的 JB 检验和 Q-Q 图中得知，残差具有“尖峰厚尾”（特征 3）。通过指定 `dist='t'`，我们强制模型使用 t-分布 来拟合残差，t-分布本身就具有厚尾特性，因此能更准确地捕捉极端事件。

结果分析

GARCH(1,1)-t 模型摘要证实了这些选择的正确性:

--- 4.3 GARCH(1,1) 模型 (BTC 对数收益率) ---

BTC GARCH(1,1)-t 模型摘要:

Constant Mean - GARCH Model Results

Dep. Variable: btc_log_return

R-squared: 0.000

Mean Model: Constant Mean

Adj. R-squared: 0.000

Vol Model: GARCH

Log-Likelihood: -5152.85

Distribution: Standardized Student's t

AIC: 10315.7

Method: Maximum Likelihood

BIC: 10344.0

No. Observations: 2129

Date: Mon, Nov 10 2025

Df Residuals: 2128

Time: 21:21:53

Df Model: 1

Mean Model

coefstd errtP>|t|95.0% Conf. Int.

mu0.10114.490e-022.2522.434e-02[1.310e-02, 0.189]

Volatility Model

coefstd errtP>|t|95.0% Conf. Int.

omega0.15598.572e-021.8196.891e-02[-1.208e-02, 0.324]

alpha[1]0.06971.827e-023.8151.363e-04[3.388e-02, 0.105]

beta[1]0.93031.845e-0250.4280.000[0.894, 0.966]

Distribution

coefstd errtP>|t|95.0% Conf. Int.

nu3.07300.20514.9819.724e-51[2.671, 3.475]

波动率聚集被捕捉:

alpha[1] (ARCH 项, $p=1.363e-04$) 和 beta[1] (GARCH 项, $p=0.000$) 均高度显著。这证明了波动率聚集效应 (GARCH 效应) 确实存在且已被模型成功捕捉。beta[1] 接近 0.9303, 表明波动率具有高度的持续性 (今天的恐慌会持续到明天)。

尖峰厚尾被捕捉:

nu (t-分布的自由度) 系数为 3.0730 且 p-value 极小 ($9.724e-51$)。nu 值越小, 尾部越厚。3.0730 是一个非常低的值, 这强有力地证明了“尖峰厚尾”特性的存在, 并且使用 t-分布是绝对必要的。

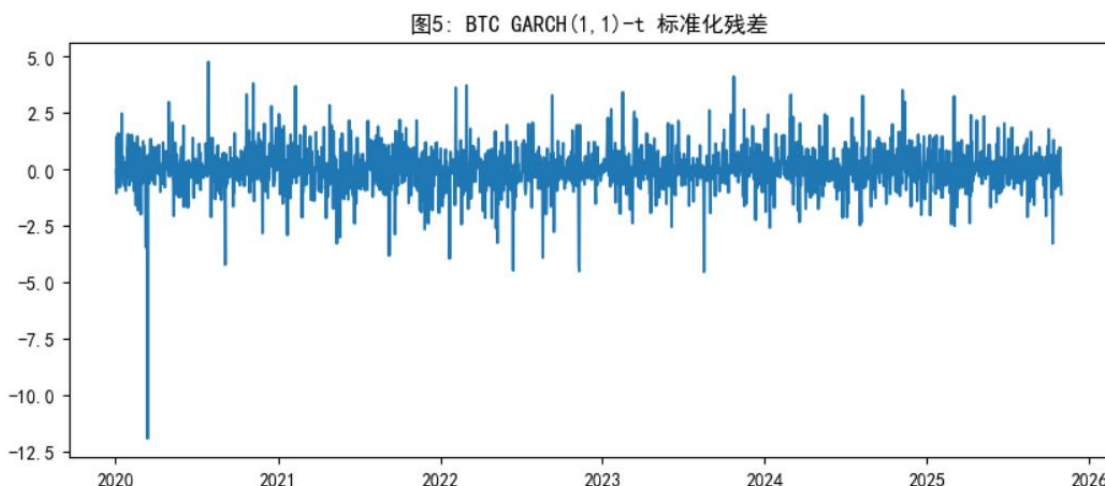


图 5 显示了 GARCH 模型的“标准化残差”。与原始收益率（图 2）相比，该图中的波动聚集现象已基本消失，序列看起来更像白噪声。

总体结论

采用了一个两步法 (ARIMA + GARCH) 来分别拟合金融序列的均值和方差。

首先使用 ARIMA(1, 1) 成功“清洗”了原始收益率中的线性自相关性。

然后，通过 ARIMA 残差的异方差性和非正态性，论证了仅使用 ARIMA 是不够的。

最后，使用了一个 GARCH(1, 1) 结合 t-分布 模型，成功地捕捉了 ARIMA 遗漏的“波动率聚集”和“尖峰厚尾”特性。

这是一个完整且逻辑严谨的建模流程。

3.3 最终模型表达式

根代码运行结果，我们将 BTC 对数收益率的最佳拟合模型表达式整理如下。

3.3.1 ARIMA(1, 0, 1) 模型

我们对平稳的 BTC 对数收益率 Y_t (即 `btc_log_return`) 拟合了一个 ARMA(1, 1) 模型（即 ARIMA(1, 0, 1)）。

均值方程：

$$Y_t = c + \phi_1 Y_{t-1} + \theta_1 \epsilon_{t-1} + \epsilon_t$$

代入 `coef` 列的估计系数：

$$Y_t = 0.0013 - 0.5954 \cdot Y_{t-1} + 0.5436 \cdot \epsilon_{t-1} + \epsilon_t$$

2. 残差分布：

$$\epsilon_t \sim \text{WN}(0, \sigma^2)$$

其中，残差方差 σ^2 的估计值为 0.0010。

3.3.2 GARCH(1,1)-t 模型

1.波动率方程

$$\sigma_t^2 = \omega + \alpha_1 a_{t-1}^2 + \beta_1 \sigma_{t-1}^2$$

带入 coef 列的估计参数:

$$\sigma_t^2 = 0.1559 + 0.0697 a_{t-1}^2 + 0.9303 \sigma_{t-1}^2$$

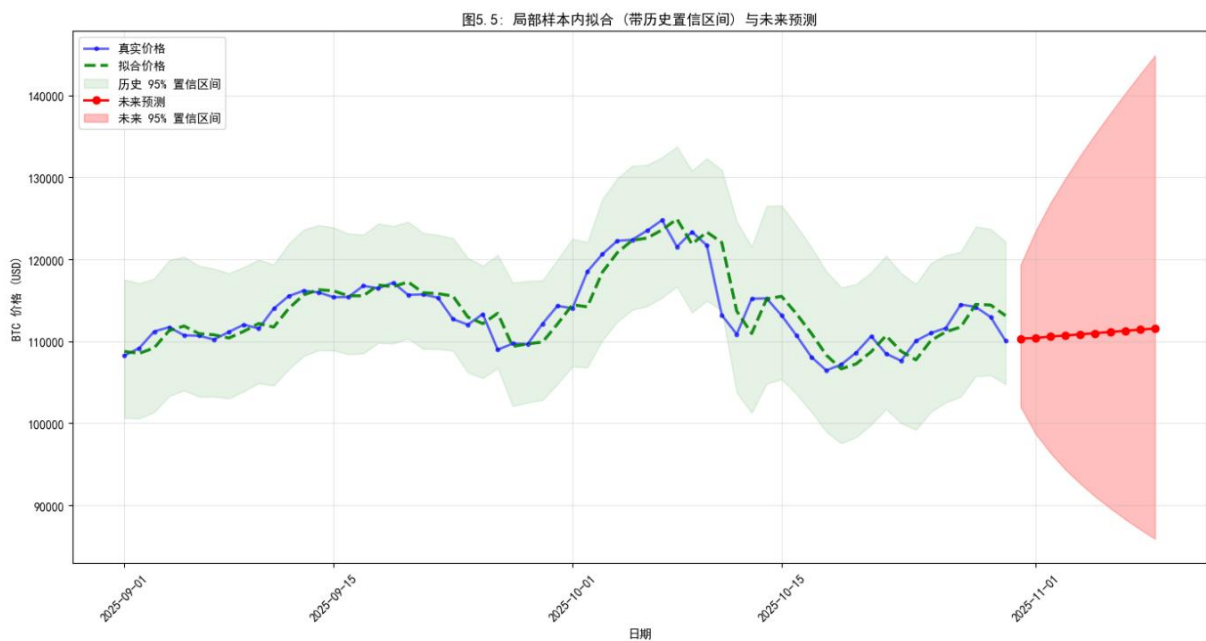
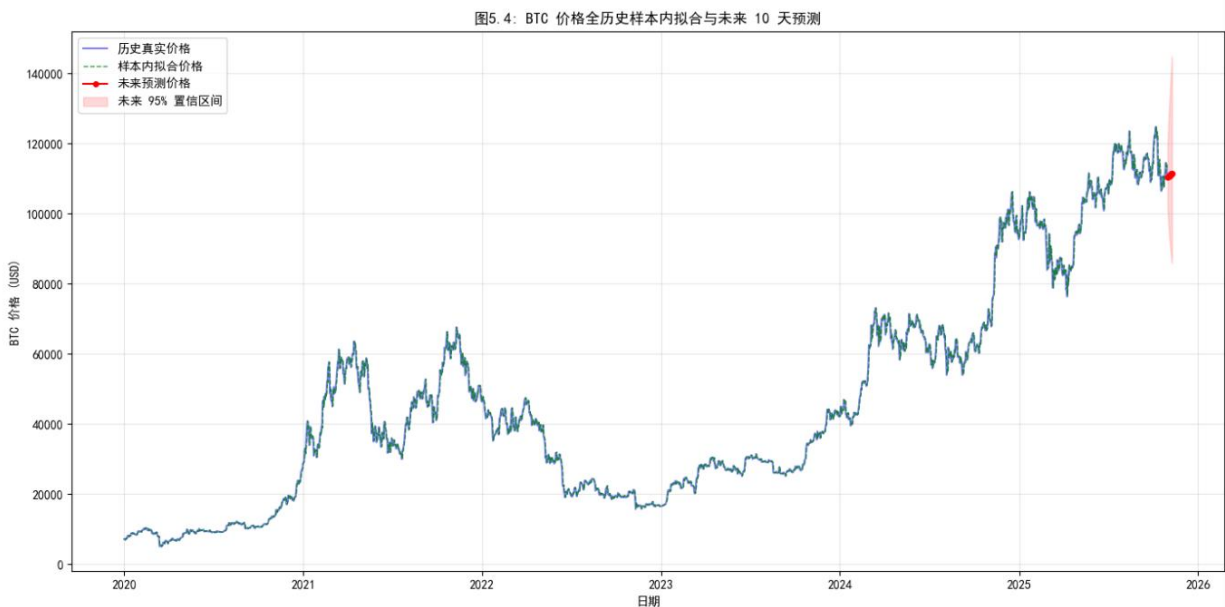
分布假设

标准化残差 ϵ_t 服从 t 分布，而不是正态分布

$$\epsilon_t \sim t(3.0730)$$

3.4 模型预测

日期 (Date)	预测价格 (PredictedPrice)	95%置信下限 (Lower95%CI)	95%置信上限 (Upper95%CI)
2025-10-31	110,331.40	102,013.99	119,326.95
2025-11-01	110,386.28	98,736.08	123,411.13
2025-11-02	110,578.50	96,378.35	126,870.86
2025-11-03	110,689.13	94,353.94	129,852.37
2025-11-04	110,848.69	92,626.32	132,655.93
2025-11-05	110,979.36	91,052.98	135,266.51
2025-11-06	111,127.55	89,627.68	137,784.80
2025-11-07	111,265.58	88,298.75	140,206.17
2025-11-08	111,409.96	87,059.49	142,571.23
2025-11-09	111,550.84	85,887.69	144,882.1



3.5 风险价值 (VaR) 回测分析 (基于 GARCH(1,1)-t 模型)

1. VaR 的概念与计算方法

风险价值 (Value at Risk, VaR) 是一种广泛使用的市场风险度量指标。它回答了这样一个问题：在正常的市场条件下，在给定的置信水平（例如 95%）下，某一投资组合在未来特定期限内（例如 1 天）可能遭受的最大损失是多少？

本报告使用 GARCH(1,1)-t 模型来计算动态 VaR。与传统的静态 VaR（假设波动率恒定）不同，GARCH-VaR 能够根据市场近期的波动情况自动调整风险预警线：

当市场平静时，预测的波动率 σ_t 较低，VaR 阈值收窄。

当市场动荡时，预测的波动率 σ_t 升高，VaR 阈值变宽。

计算公式为：

$$VaR_t^\alpha = \mu_t + \sigma_t \cdot F^{-1}(\alpha, \nu)$$

其中：

$\alpha = 5\%$ （对应 95% 置信水平）。

μ_t 是模型预测的条件均值（本例中近似为常数）。

σ_t 是 GARCH 模型预测的条件波动率。

$F^{-1}(\alpha, \nu)$ 是自由度为 ν 的 t-分布的 5% 分位点（这是一个负数）。

2. 回测结果分析 (图 8)

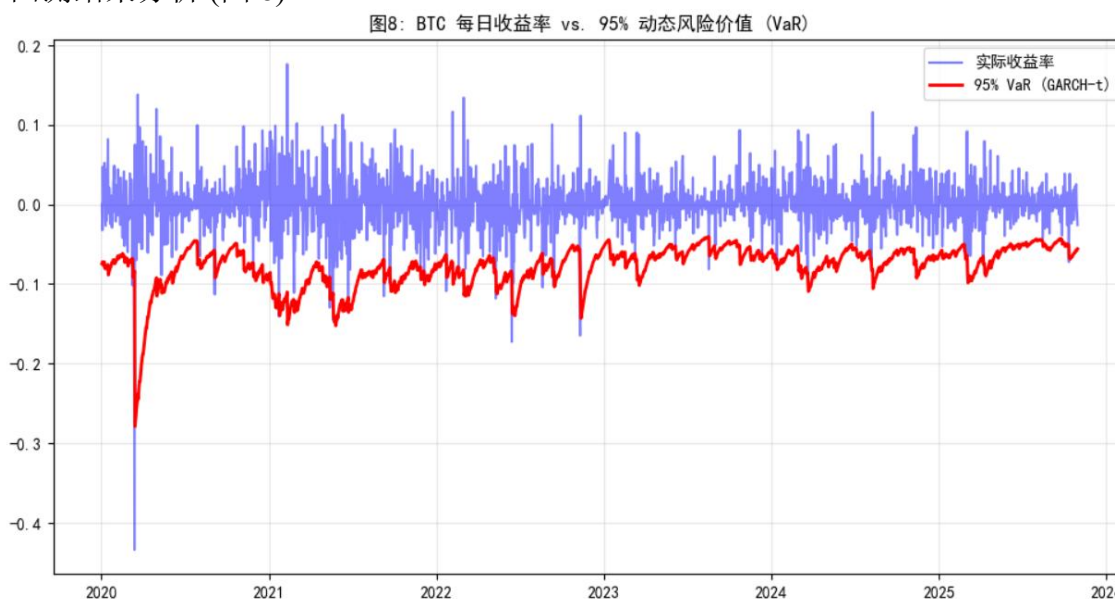


图 8 直观地展示了 GARCH-VaR 模型在过去几年中的表现：

蓝线：BTC 每日的实际对数收益率。

红线：模型逐日计算出的 95% VaR 阈值。它构成了一条动态的“风险底线”。

观察要点：

动态适应性：红线并非一条水平直线，而是随着蓝线的波动而剧烈起伏。

在 2020 年 3 月（市场大跌）和 2021 年中旬（高位震荡）等高波动时期，红线迅速向下探底，扩大了风险预警范围。

在 2023 年初等相对平静期，红线则向上收缩，表明此时市场风险较低。这证明 GARCH 模型成功捕捉到了市场的波动率聚集特性。

突破情况 (Exceptions): 当蓝线跌破红线时，称为一次“VaR 突破”（即实际损失超过了预期最大损失）。从图中可以看到，这种情况确实偶尔发生，但并不频繁。

3. 定量回测统计

根据控制台输出的统计数据：

总观测天数：2129 天

理论突破率：5.00%（由 95% 置信水平决定）

实际突破次数：32 次

实际突破率： $32 \div 2129 \approx 1.50\%$

3.1 结果解读：模型过于保守

理想情况下，实际突破率应非常接近理论值 5%。

结果 (1.50%) 显著低于理论值 (5.00%)。

这意味着 GARCH-VaR 模型高估了市场风险。它划定的“安全线”（红线）画得太低了，以至于实际损失很少能触及它。

从风险控制的角度看，这可能被视为一件“好事”，因为模型非常谨慎，提供了额外的安全边际。

从资本效率的角度看，这可能是一件“坏事”。如果金融机构依据此模型计提风险准备金，他们可能会留存过多的闲置资金，导致资金利用率低下。

3.2 可能的原因

造成实际突破率偏低的主要原因可能在于分布假设：

我们使用了 t-分布来捕捉厚尾特征。模型拟合出的自由度 $\nu \approx 3.07$ 非常低，这意味着模型假设市场发生极端黑天鹅事件的概率非常高。

有可能真实的 BTC 市场虽然比正态分布“尾巴厚”，但并没有 $t(3.07)$ 分布预测的那么“厚”。因此，模型为了防范那些理论上存在但实际上没发生的极端暴跌，把 VaR 线设得过低了。

4. 未来一日风险预测

模型对下一个交易日给出的风险提示是：

95% VaR: -5.7166%

解读：根据当前的市场波动率水平，模型有 95% 的把握认为，明天 BTC 的跌幅不会超过 5.72%。如果明天跌幅超过这个值，那将是一个小概率（<5%）的极端风险事件。

4. 多元时间序列

4.1 协整性检验

1. 滞后阶数选择

在进行多元时间序列建模之前，首先需要确定最优滞后阶数。我们分别对收益率序列（df_returns）和价格序列（df_prices）拟合了向量自回归 (VAR) 模型，并基于赤池信息准则 (AIC) 选择最优阶数。

收益率序列 (VAR): 最优滞后阶数为 10。这表明 BTC 和 ETH 收益率之间的相互影响可追溯至过去 10 个交易日。

价格序列 (VAR): 最优滞后阶数为 8。此结果用于确定后续 Johansen 协整检验中的滞后差分阶数。

2. 协整检验 (Johansen Test)

为了探究 BTC 和 ETH 价格之间是否存在长期稳定的均衡关系，我们对非平稳的价格序列进行了 Johansen 协整检验。

原假设 ($H_0: r = 0$): 不存在协整关系。

迹统计量 (Trace Stat) = 15.1190

5% 临界值 (95% Crit) = 18.3985

结论: 由于迹统计量小于临界值 ($15.1190 < 18.3985$)，我们无法拒绝原假设。这表明 BTC 和 ETH 价格之间不存在协整关系。

原假设 ($H_0: r \leq 1$): 至多存在 1 个协整关系。

迹统计量 (Trace Stat) = 3.7944

5% 临界值 (95% Crit) = 3.8415

结论: 同样无法拒绝原假设，进一步确认了不存在多个协整关系的结论。

分析: 协整检验结果表明，BTC 和 ETH 的价格在长期走势上并没有被绑定在一起。它们各自可能有独立的随机游走趋势，不存在一个长期会将它们拉回固定比例的均衡机制。因此，建立向量误差修正模型 (VECM) 并不合适。

4.2 格兰杰因果检验 (Granger Causality Test)

虽然价格之间没有长期协整关系，我们进一步考察了它们的收益率在短期内是否存在预测关系。我们在平稳的收益率序列上进行了格兰杰因果检验，滞后阶数设定为 10。

检验 1: ETH 是否格兰杰导致 BTC ($H_0: \text{ETH} \nrightarrow \text{BTC}$)

p-value = 0.000003

结论: p 值远小于 0.05，强烈拒绝原假设。这表明 ETH 的过去收益率对预测 BTC 的当前收益率有显著的统计意义。

检验 2: BTC 是否格兰杰导致 ETH ($H_0: \text{BTC} \nrightarrow \text{ETH}$)

p-value = 0.008054

结论: p 值小于 0.05，拒绝原假设。这表明 BTC 的过去收益率同样对预测 ETH 的当前收益率有显著作用。

分析: 格兰杰因果检验揭示了 BTC 和 ETH 市场之间存在显著的双向因果关系。这意味着

两个市场的信息流动是高度双向互动的，一个市场的价格波动会迅速传导并影响另一个市场。这种紧密的短期联动性是加密货币市场高相关性的典型特征。

4.3 VAR（向量自回归）模型

1. 模型选择依据

在确立最终的多元时间序列模型时，我们依据了前序检验的关键结果：

无长期协整关系：Johansen 检验表明 BTC 和 ETH 的价格序列不存在长期稳定的均衡关系，因此不具备建立向量误差修正模型（VECM）的前提条件。

存在短期因果关系：格兰杰因果检验揭示了 BTC 和 ETH 的对数收益率在短期内存在显著的双向预测能力。

数据平稳性：对数收益率序列已被证实是平稳的 (I(0))。

基于上述特征，向量自回归 (VAR) 模型成为最合适的选择。VAR 模型专门用于建模多个平稳时间序列之间的相互依赖关系，能够有效捕捉两个市场在短期内的动态冲击传导机制。

2. 模型原理简述

向量自回归 (VAR) 模型是单变量自回归 (AR) 模型在多变量系统中的推广。其核心思想是不预先设定变量的内生性或外生性，而是将系统中的所有变量都视为内生变量。

在我们的双变量系统中，BTC 和 ETH 在 t 时刻的收益率不仅取决于它们各自过去的表现，还取决于对方过去的表现。一个 p 阶的 VAR 模型可以表示为：

$$Y_t = C + A_1 Y_{t-1} + A_2 Y_{t-2} + \dots + A_p Y_{t-p} + \epsilon_t$$

其中：

$Y_t = (\text{BTC_Ret}_t, \text{ETH_Ret}_t)'$ 是包含两个内生变量的列向量。

$A_1, \dots, A_p$ 是 2×2 的系数矩阵，捕捉了滞后期的交叉影响。

ϵ_t 是误差项向量，包含了当前时刻无法被过去信息解释的共同冲击（同期相关性）。

3 模型拟合结果分析

我们对 BTC 和 ETH 的日对数收益率序列拟合了 10 阶向量自回归模型 (VAR(10))。模型整体稳定性检验通过（所有特征根的模均小于 1），确保了分析的有效性。

--- 5.4 拟合 VAR 模型 (在收益率上) ---
正在拟合 VAR(10) 模型...

VAR 模型摘要:

Summary of Regression Results

=====

Model: VAR
Method: OLS
Date: Mon, 10, Nov, 2025
Time: 21:55:41

No. of Equations: 2.00000 BIC: -14.1649
Nobs: 2119.00 HQIC: -14.2360
Log likelihood: 9155.09 FPE: 6.30306e-07
AIC: -14.2771 Det(Omega_mle): 6.17996e-07

Results for equation btc_log_return

=====

	coefficient	std. error	t-stat	prob
const	0.001195	0.000700	1.708	0.088
L1.btc_log_return	0.047508	0.038380	1.238	0.216
L1.eth_log_return	-0.076008	0.028296	-2.686	0.007
L2.btc_log_return	0.025005	0.038377	0.652	0.515
L2.eth_log_return	0.005699	0.028351	0.201	0.841
L3.btc_log_return	0.045888	0.038336	1.197	0.231
L3.eth_log_return	-0.035128	0.028352	-1.239	0.215
L4.btc_log_return	-0.027459	0.038185	-0.719	0.472
L4.eth_log_return	0.068248	0.028297	2.412	0.016
L5.btc_log_return	0.097040	0.038184	2.541	0.011
L5.eth_log_return	-0.075686	0.028307	-2.674	0.008
L6.btc_log_return	-0.078724	0.038246	-2.058	0.040
L6.eth_log_return	0.065472	0.028358	2.309	0.021
L7.btc_log_return	0.047977	0.038272	1.254	0.210
L7.eth_log_return	-0.069932	0.028422	-2.460	0.014
L8.btc_log_return	0.014765	0.038275	0.386	0.700
L8.eth_log_return	-0.053090	0.028472	-1.865	0.062
L9.btc_log_return	0.077222	0.038270	2.018	0.044
L9.eth_log_return	-0.025891	0.028479	-0.909	0.363
L10.btc_log_return	-0.060833	0.038244	-1.591	0.112
L10.eth_log_return	0.066516	0.028468	2.337	0.019

=====

Results for equation eth_log_return

=====

	coefficient	std. error	t-stat	prob
const	0.001428	0.000948	1.506	0.132
L1.btc_log_return	0.002552	0.052012	0.049	0.961
L1.eth_log_return	-0.057384	0.038346	-1.496	0.135
L2.btc_log_return	0.026041	0.052007	0.501	0.617
L2.eth_log_return	0.028914	0.038420	0.753	0.452
L3.btc_log_return	0.085481	0.051952	1.645	0.100
L3.eth_log_return	-0.043756	0.038422	-1.139	0.255
L4.btc_log_return	-0.008108	0.051747	-0.157	0.875
L4.eth_log_return	0.052542	0.038347	1.370	0.171
L5.btc_log_return	0.138802	0.051745	2.682	0.007
L5.eth_log_return	-0.110108	0.038360	-2.870	0.004
L6.btc_log_return	-0.164388	0.051829	-3.172	0.002
L6.eth_log_return	0.126283	0.038430	3.286	0.001
L7.btc_log_return	0.011628	0.051865	0.224	0.823
L7.eth_log_return	-0.033549	0.038517	-0.871	0.384
L8.btc_log_return	0.030462	0.051869	0.587	0.557
L8.eth_log_return	-0.062563	0.038584	-1.621	0.105
L9.btc_log_return	0.092368	0.051862	1.781	0.075
L9.eth_log_return	-0.046166	0.038594	-1.196	0.232
L10.btc_log_return	-0.050553	0.051826	-0.975	0.329
L10.eth_log_return	0.094916	0.038578	2.460	0.014

=====

Correlation matrix of residuals

	btc_log_return	eth_log_return
btc_log_return	1.000000	0.822717
eth_log_return	0.822717	1.000000

3.(1)BTC 收益率方程 (btc_log_return)

BTC 当期的收益率主要受自身滞后 5, 6, 9 期, 以及 ETH 滞后 1, 4, 5, 6, 7, 10 期的显著影响。

$$\begin{aligned} R_{BTC,t} = & 0.0012 \\ & +0.097R_{BTC,t-5} - 0.079R_{BTC,t-6} + 0.077R_{BTC,t-9} \\ & -0.076R_{ETH,t-1} + 0.068R_{ETH,t-4} - 0.076R_{ETH,t-5} \\ & +0.065R_{ETH,t-6} - 0.070R_{ETH,t-7} + 0.067R_{ETH,t-10} \\ & + \epsilon_{BTC,t} \end{aligned}$$

(注: 常数项 0.0012 的 p 值为 0.088, 接近显著, 此处予以保留)

此方程描述了 BTC 当期收益率如何受自身及 ETH 过去 10 天收益率的影响。

显著的跨市场影响 (ETH -> BTC):

L1.eth_log_return (滞后 1 期 ETH): 系数为 -0.0760, t 统计量为 -2.686 (p=0.007)。这表明昨天 ETH 的收益率对今天 BTC 的收益率有显著的负向修正作用。

L4, L5, L6, L7, L10 期的 ETH 滞后项也显示出统计显著性 (p < 0.05)。这揭示了 ETH 市场对 BTC 市场存在复杂且持续的溢出效应。

自身滞后影响 (BTC -> BTC):

BTC 自身的滞后项中, 只有 L5 (p=0.011), L6 (p=0.040), L9 (p=0.044) 表现出显著性。这表明 BTC 市场的短期记忆性相对较弱, 更多受即期市场情绪和跨市场信息的影响。

(2) ETH 收益率方程 (eth_log_return)

ETH 当期的收益率主要受 BTC 滞后 5, 6 期, 以及自身滞后 5, 6, 10 期的显著影响。

$$\begin{aligned} R_{ETH,t} = & 0.0014 \\ & +0.139R_{BTC,t-5} - 0.164R_{BTC,t-6} \\ & -0.110R_{ETH,t-5} + 0.126R_{ETH,t-6} + 0.095R_{ETH,t-10} \\ & + \epsilon_{ETH,t} \end{aligned}$$

(注: 常数项 0.0014 的 p 值为 0.132, 不显著, 但为保持方程完整性选择保留)

此方程描述了 ETH 当期收益率如何受自身及 BTC 过去收益率的影响。

显著的跨市场影响 (BTC -> ETH):

L5.btc_log_return: 系数为 0.1388 (p=0.007)。

L6.btc_log_return: 系数为 -0.1644 (p=0.002)。

这表明 BTC 市场的信息传导到 ETH 市场存在一定的时滞, 5-6 天前的 BTC 波动会对今天的 ETH 价格产生显著影响。

自身滞后影响 (ETH -> ETH):

ETH 自身的 L5 (p=0.004), L6 (p=0.001), L10 (p=0.014) 滞后项显著。与 BTC 类似, ETH 市

场也表现出中期（5-6 天）的显著记忆性。

3.(3) 残差相关性分析

两个方程的误差项 $\epsilon_{BTC,t}$ 和 $\epsilon_{ETH,t}$ 并不是独立的，它们之间存在很强的同期相关性：

$$\text{Corr}(\epsilon_{BTC,t}, \epsilon_{ETH,t}) = 0.8227$$

这个高相关系数表明，除了上述方程捕捉到的滞后传导效应外，每天还有大量的新信息（如宏观新闻，政策调控等）会同时推动这两种资产的价格向同一方向大幅波动。

4.4 脉冲响应分析（IRF）

4.4.1 分析目的与原理

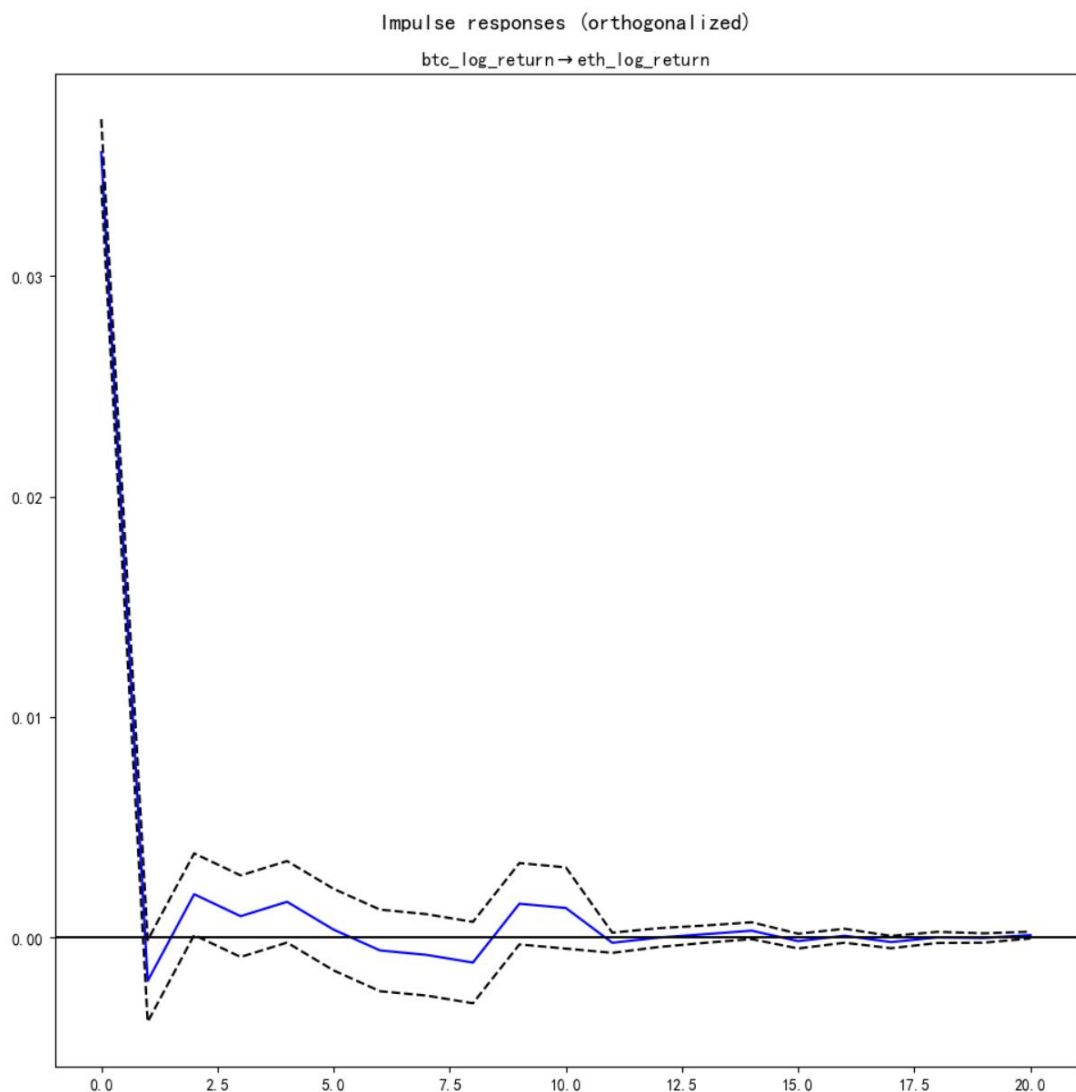
1. 分析目的 (Why): 虽然 VAR 模型提供了变量间动态关系的精确数学描述，但其复杂的滞后系数矩阵难以直观解读。脉冲响应函数 (IRF) 的目的在于将这些复杂的系数量化为直观的图形：它模拟当系统中的某一个变量突然受到一个单位（通常为一倍标准差）的外部冲击时，其他变量在未来若干期内的动态反应路径。这有助于我们理解市场冲击的传导方向、速度和持续时间。

2. 原理简述 (How): 在我们的 VAR 模型中，残差项之间存在高达 0.82 的同期相关性。这意味着在现实中，BTC 和 ETH 的冲击往往是同时发生的，很难单独剥离出“纯粹”的 BTC 或 ETH 冲击。为了解决这个问题，我们采用了正交化脉冲响应 (Orthogonalized IRF)，其核心是通过 Cholesky 分解将相关的残差转化为相互独立的结构性冲击。

变量排序的重要性：Cholesky 分解要求我们人为设定变量的“因果顺序”。我们将排序设定为 [BTC 收益率, ETH 收益率]。

经济学含义：这一排序隐含的假设是，在冲击发生的当天 (第 0 天)，BTC 的冲击可以瞬间传导给 ETH，但 ETH 的冲击无法在当天反向影响 BTC（只能从第二天开始影响）。鉴于 BTC 在加密市场中的主导地位，这一假设是合理的。

4.4.2 BTC 冲击对 ETH 的影响



当期反应 (第 0 天):

当 BTC 收益率受到一个正向标准差冲击时，ETH 收益率在当天 (Lag 0) 就表现出极强且显著的正向反应（响应值约为 0.035，远高于 0 水平线）。

这证实了 BTC 作为市场风向标的地位，其价格波动会迅速、强烈地传导至 ETH 市场。

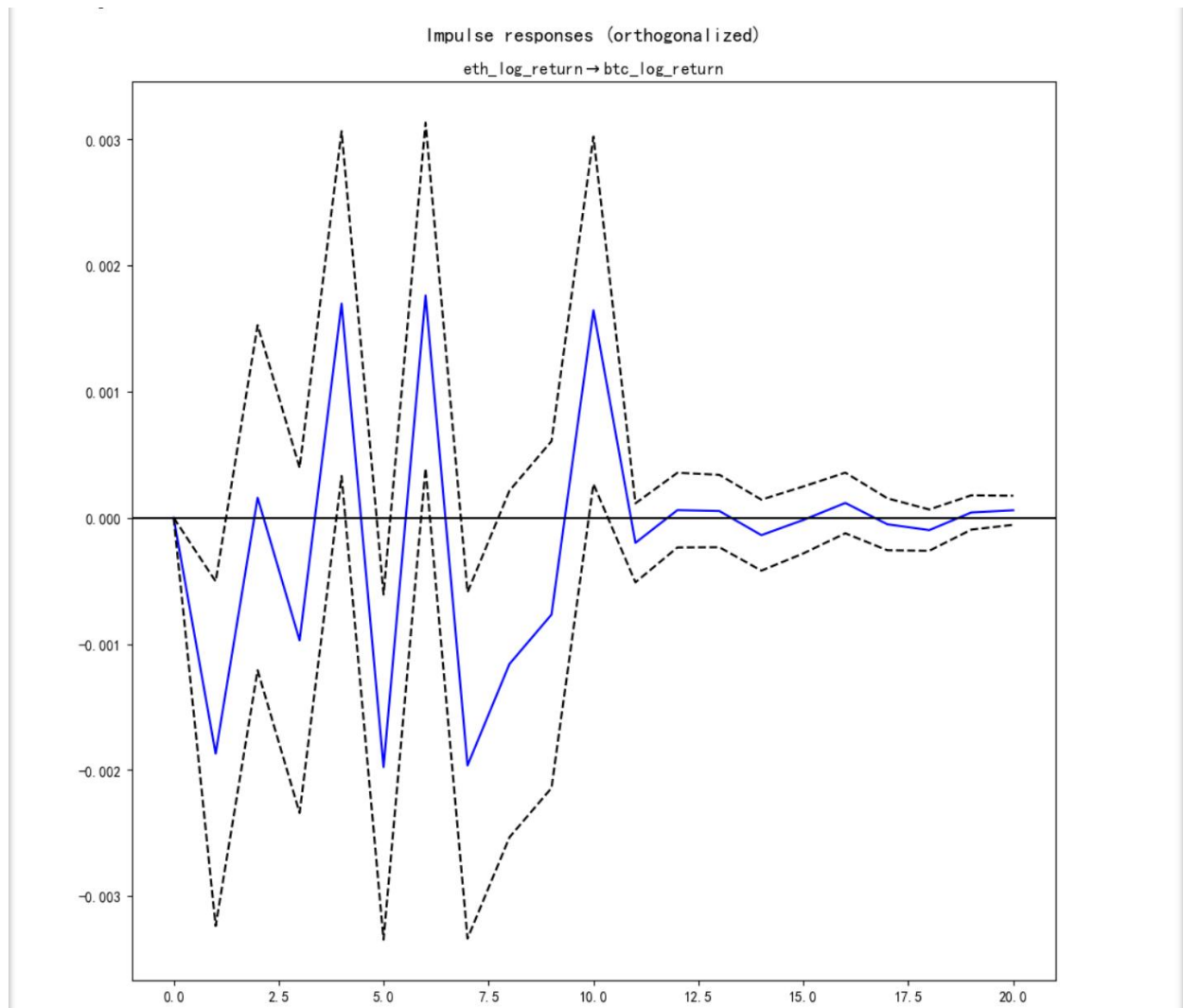
后续反应 (第 1-20 天):

这种冲击的影响衰减得非常快。在第 1 天，ETH 的响应迅速回落至接近 0 的水平。

在随后的几天里（如第 3-5 天和第 9 天），ETH 的响应在 0 附近小幅波动，但 95% 置信区间（虚线）大多包含了 0 水平线，说明这些后续反应在统计上大多不再显著。

结论：BTC 对 ETH 的冲击是“脉冲式”的——来得猛烈，去得也快，主要集中在当天。

4.4.3 ETH 冲击对 BTC 的影响



当期反应 (第 0 天):

由于 Cholesky 排序的设定, ETH 对 BTC 的当期 (Lag 0) 影响被强制为 0。

后续反应 (第 1-20 天):

从第 1 天开始, BTC 对 ETH 前一天的冲击开始做出反应。

值得注意的是, BTC 的响应呈现出一种“震荡”模式, 在正负之间交替波动 (例如第 1 天为负, 第 2 天回升, 第 3 天又下降)。

然而, 观察置信区间可以发现, 绝大多数时候 0 水平线都包含在区间内。这意味着 ETH 的冲击对 BTC 的后续影响在统计上并不显著。

结论: 相比之下, ETH 对 BTC 的溢出效应要弱得多, 且缺乏明确的方向性。

4.4.4 总结

脉冲响应分析清晰地揭示了加密货币市场中的不对称信息传导机制：

BTC 主导地位：BTC 市场的波动对 ETH 市场具有极其显著、迅速且强烈的正向引导作用。

ETH 跟随特征：ETH 更多是作为 BTC 价格波动的“接收者”，其自身波动对 BTC 的反向影响相对微弱且不明确。

4.5 方差分解分析 (Variance Decomposition, FEVD)

4.5.1 分析目的与原理

1). 分析目的 (Why): 脉冲响应函数展示了冲击传导的“路径”，而方差分解则进一步量化了这种冲击的“重要性”。它的目的是将系统中每个变量的预测误差方差分解为各个变量冲击所贡献的比例。简单来说，它告诉我们：在未来某一时刻，某个币种价格波动的百分之多少是由它自己的“新消息”造成的，又有百分之多少是由另一个币种的“新消息”造成的。这能直观地反映出各市场在系统中的相对重要性和信息贡献度。

2). 原理简述 (How): 方差分解同样基于 VAR 模型的正交化残差（通常使用 Cholesky 分解）。对于一个 k 步向前的预测，其实际值与预测值之间的误差是由这 k 期间发生的所有结构性冲击累积而成的。方差分解就是计算在这些预测误差的总方差中，由第 j 个变量的结构性冲击所引起的方差所占的百分比。

如果一个变量的预测方差几乎全由自身冲击解释，说明它是相对外生的（主导者）。

如果一个变量的预测方差很大一部分能由其他变量的冲击解释，说明它是相对内生的（跟随者）。

4.5.2 结果分析 (基于未来 10 天)

我们对未来 10 天的预测方差进行了分解，结果如下：


```

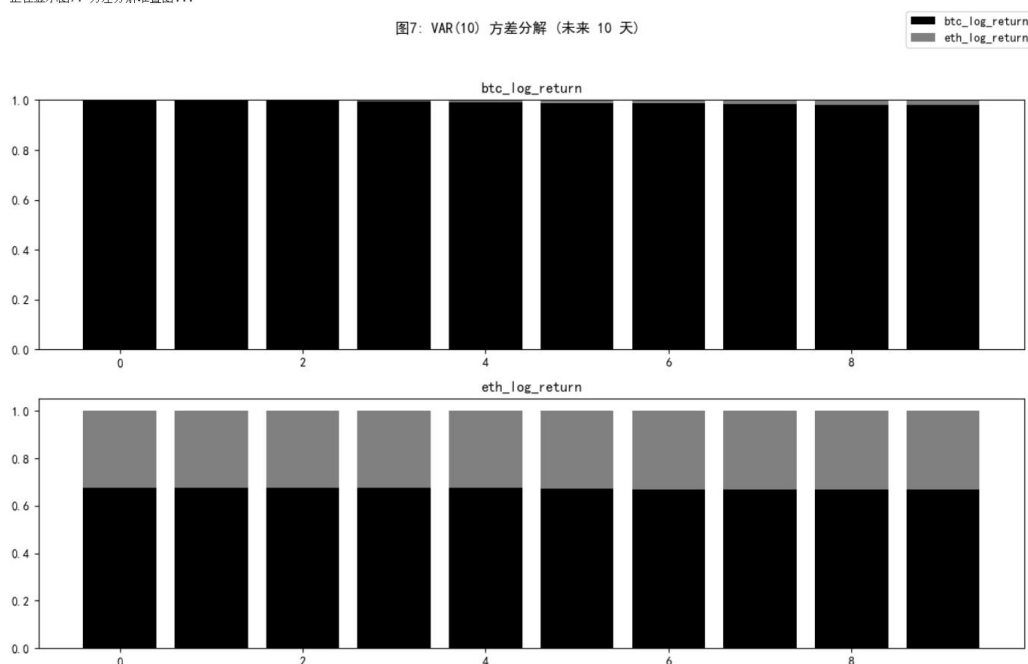
方差分解结果 (未来 10 天):
FEVD for btc_log_return
  btc_log_return  eth_log_return
0      1.000000      0.000000
1      0.996588      0.003412
2      0.996568      0.003432
3      0.995652      0.004348
4      0.992884      0.007116
5      0.989143      0.010857
6      0.986191      0.013809
7      0.982561      0.017439
8      0.981329      0.018671
9      0.980828      0.019172

FEVD for eth_log_return
  btc_log_return  eth_log_return
0      0.676864      0.323136
1      0.676808      0.323192
2      0.677251      0.322749
3      0.676887      0.323113
4      0.676744      0.323256
5      0.673731      0.326269
6      0.670047      0.329953
7      0.669768      0.330232
8      0.669455      0.330545
9      0.669384      0.330616

```

None
正在显示图7: 方差分解堆叠图...

图7: VAR(10) 方差分解 (未来 10 天)



1). BTC 收益率的方差分解 (FEVD for btc_log_return)

主导地位确认: BTC 表现出极强的外生性。

在第 0 天（当期），BTC 的价格波动 100% 由其自身解释（这是 Cholesky 排序将其置于首位的结果）。

即使到了第 10 天（Lag 9），BTC 自身冲击对预测方差的贡献率依然高达 98.08%，而 ETH 的贡献率仅为 1.92%。

结论: 这表明 BTC 市场基本上是“自驱动”的，其价格变动极少受到 ETH 市场特定信息的影响。它是整个加密货币市场信息的主要来源。

2). ETH 收益率的方差分解 (FEVD for eth_log_return)

显著的跟随特征: ETH 表现出很强的内生性。

在第 0 天（当期），ETH 的价格波动中就有高达 67.69% 是由 BTC 的同期冲击解释的，只有 32.31% 是由 ETH 自身的特有冲击解释的。

随着时间推移到第 10 天（Lag 9），这一比例基本保持稳定，BTC 的贡献率维持在 66.94% 左右，ETH 自身的贡献率约为 33.06%。

结论：这意味着 ETH 价格的大部分波动（约三分之二）实际上并不是由 ETH 自身的特定新闻（如网络升级、应用生态等）驱动的，而是简单地跟随 BTC 的市场大势。只有约三分之一的波动真正反映了 ETH 自身的独立市场信息。

4.5.3 总结

方差分解的结果提供了强有力的证据，支持了“BTC 是市场领导者，ETH 是市场跟随者”这一假说：

BTC 的价格动态几乎完全由自身主导。

ETH 的价格动态在很大程度上（>67%）受 BTC 主导。这一发现对于风险管理和投资组合构建具有重要意义，提示投资者在配置加密资产时，需充分考虑这种高度的单向依赖性。

5. 总结与展望

5.1 主要研究结论

本研究通过对 2020 年至 2025 年比特币（BTC）与以太坊（ETH）日度价格数据的系统性时间序列分析，得出以下核心结论：

市场风险特征显著：BTC 与 ETH 的收益率序列均呈现出极其显著的“尖峰厚尾”和“波动聚集”特征。传统的正态分布假设严重低估了市场发生极端价格波动的概率。引入 t-分布的 GARCH(1,1) 模型成功捕捉了这些特征，证明了加密市场风险具有高度的时变性和不可预测性。

缺乏长期价格锚定：协整检验表明，尽管 BTC 和 ETH 价格走势在视觉上高度相关，但两者之间并不存在长期稳定的均衡关系。这意味着它们的价格可以在长期内各自独立漂移，不存在一种必然将它们拉回固定比率的市场力量。

短期内 BTC 具有绝对主导权：格兰杰因果检验、脉冲响应分析和方差分解一致确认了市场存在显著的不对称信息传导机制。BTC 是市场的“价格发现者”，其波动能迅速、强烈地正向传导至 ETH；而 ETH 更多扮演“跟随者”角色，其自身信息对 BTC 的反向影响微乎其微（解释力度不足 2%）。

5.2 投资启示与建议

基于上述实证结论，对加密资产投资者和风险管理者提出以下建议：

风险管理需重视极端尾部风险：鉴于收益率的厚尾特性，使用传统的方差或标准差来衡量风险是不够的。建议采用基于 GARCH-t 模型的动态风险价值（VaR）或预期亏损（ES）作为更可靠的风险监控指标，以防范突发的市场黑天鹅事件。

以 BTC 为核心的市场监测指标：由于 BTC 对整个市场具有强大的引导作用，投资者在交易 ETH 或其他山寨币时，必须首先密切关注 BTC 的动向。BTC 的重大价格突破或异常波动往往是整个加密板块变盘的前兆信号。

跨品种套利需谨慎：由于 BTC 和 ETH 之间不存在长期协整关系，基于均值回归原理的配对交易策略（Pairs Trading）可能面临价差长期无法回归的风险。投资者在设计相关套利策略时需格外谨慎，设置严格的止损线。

5.3 研究局限性与未来展望

本研究虽然得出了一系列有价值的结论，但仍存在一定局限性，未来可在以下方向进行拓展：

高频数据的应用：本研究仅使用了日度收盘价数据，可能掩盖了日内更微观、更迅速的信息传导过程。未来可引入分钟级甚至逐笔交易数据，更精细地考察市场微观结构和极短期的冲击效应。

动态相关性的刻画：VAR 模型假设资产间的相关性是恒定的。然而在现实中，特别是在市场极端恐慌时期，资产相关性往往会急剧上升。未来可采用 DCC-GARCH（动态条件相关 GARCH）等模型，深入研究市场相关结构的时变特征及其在不同市场环境下的差异。

外部宏观因素的纳入：加密市场已日益融入全球金融体系，受美联储货币政策、通胀数据、地缘政治等宏观因素影响显著。未来的模型可尝试纳入这些外部宏观变量，构建更全面的多因子预测模型，以提高对市场大趋势研判的准确性。

附录：python 代码

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
import itertools
from statsmodels.tsa.stattools import adfuller, grangercausalitytests, kpss
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.api import VAR
from statsmodels.tsa.vector_ar.vecm import VECM, coint_johansen
from statsmodels.stats.diagnostic import acorr_ljungbox
from arch import arch_model
from scipy.stats import t
import warnings

warnings.filterwarnings("ignore")
plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号

# --- 1. 数据导入和预处理 ---
file_name = r"C:\study\时间序列分析\结课作业\data.xlsx"
df = pd.read_excel(file_name)
df['date'] = pd.to_datetime(df['date'])
df.set_index('date', inplace=True)
df.sort_index(inplace=True)
df_prices = df[['btc_close', 'eth_close']].copy().dropna()

print("数据加载成功，预处理完成。")
print("数据概览:")
print(df_prices.head())
print("\n 数据信息:")
df_prices.info()

# --- 2. 数据基本统计分析 ---
print("\n--- 2. 描述性统计 ---")
print(df_prices.describe())

# 绘制价格时序图
print("正在显示图 1: 原始价格时序图 (BTC vs ETH)...")
df_prices.plot(subplots=True, figsize=(10, 8))
plt.suptitle('图 1: 原始价格时序图 (BTC vs ETH)')
plt.show()

# 计算对数收益率
df_returns = np.log(df_prices).diff().dropna()
df_returns.columns = ['btc_log_return', 'eth_log_return']

print("正在显示图 2: 对数收益率时序图 (BTC vs ETH)...")
df_returns.plot(subplots=True, figsize=(10, 8))
plt.suptitle('图 2: 对数收益率时序图 (BTC vs ETH)')
plt.show()

# 绘制对数收益率直方图
print("正在显示图 3: 对数收益率分布直方图...")
plt.figure(figsize=(12, 5))
```

```

plt.subplot(1, 2, 1)
sns.histplot(df_returns['btc_log_return'], kde=True, bins=30)
plt.title('BTC 对数收益率分布')
plt.subplot(1, 2, 2)
sns.histplot(df_returns['eth_log_return'], kde=True, bins=30)
plt.title('ETH 对数收益率分布')
plt.suptitle('图 3: 对数收益率分布直方图')
plt.show()

# --- 改进 1: 检验尖峰厚尾 (Skewness, Kurtosis) 和 Q-Q 图 ---
print("\n--- 2.1 收益率分布检验 (尖峰厚尾) ---")
print(f"BTC 收益率 偏度 (Skewness): {df_returns['btc_log_return'].skew():.4f} (正态=0)")
print(f"BTC 收益率 峰度 (Kurtosis): {df_returns['btc_log_return'].kurtosis():.4f} (正态=0, 金融>0)")
print(f"ETH 收益率 偏度 (Skewness): {df_returns['eth_log_return'].skew():.4f}")
print(f"ETH 收益率 峰度 (Kurtosis): {df_returns['eth_log_return'].kurtosis():.4f}")

print("\n正在显示图 3.1: 收益率 Q-Q 图 (对比正态分布)...")
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
sm.qqplot(df_returns['btc_log_return'].dropna(), line='s', ax=axes[0])
axes[0].set_title('BTC 收益率 Q-Q 图')
sm.qqplot(df_returns['eth_log_return'].dropna(), line='s', ax=axes[1])
axes[1].set_title('ETH 收益率 Q-Q 图')
plt.suptitle('图 3.1: Q-Q 图 (证明尖峰厚尾特性)')
plt.show()

# --- 3. 平稳性检验和白噪声检验 ---
print("\n--- 3. 平稳性检验 (ADF) 与白噪声检验 ---")

# --- 检验函数定义 ---

# ADF 检验
def run_adf_test(series, name):
    """
    ADF 检验
    H0 (原假设): 序列存在单位根 (非平稳)
    """
    result = adfuller(series)
    print(f'序列 "{name}" 的 ADF 检验 (H0: 存在单位根):')
    print(f' ADF 统计量: {result[0]:.4f}, p-value: {result[1]:.4e}')
    if result[1] > 0.05:
        print(f' 结论: 序列非平稳 (p > 0.05)')
    else:
        print(f' 结论: 序列平稳 (p <= 0.05)')
    return result[1]

# 白噪声检验 (Ljung-Box)
def run_ljung_box_test(series, name, lags=10):
    """
    Ljung-Box 白噪声检验
    H0 (原假设): 序列是白噪声 (即序列值彼此独立, 无自相关)
    """
    # Ljung-Box test
    result_df = acorr_ljungbox(series, lags=[lags], return_df=True)
    p_value = result_df.iloc[0]['lb_pvalue']
    print(f'序列 "{name}" 的 Ljung-Box 白噪声检验 (H0: 序列为白噪声):')

```

```

        print(f'    L-B 统计量 (lag={lags}): {result_df.iloc[0]["lb_stat"]:.4f}, p-value: {p_value:.4e}')

    if p_value <= 0.05:
        print(f'    结论: 序列存在自相关 (非白噪声, 可建模) (p <= 0.05)')
    else:
        print(f'    结论: 序列为白噪声 (无信息) (p > 0.05)')
    return p_value

# --- 3.1 原始价格检验 ---
print("\n--- 3.1 原始价格平稳性检验 (预期: 非平稳) ---")
p_btc_price_adf = run_adf_test(df_prices['btc_close'], 'BTC 价格')
print("----")
p_eth_price_adf = run_adf_test(df_prices['eth_close'], 'ETH 价格')

# --- 3.2 对数收益率检验 ---
print("\n--- 3.2 对数收益率平稳性与白噪声检验 (预期: 平稳, 非白噪声) ---")
p_btc_return_adf = run_adf_test(df_returns['btc_log_return'], 'BTC 对数收益率')
p_btc_return_lb = run_ljung_box_test(df_returns['btc_log_return'], 'BTC 对数收益率')

print("----")
p_eth_return_adf = run_adf_test(df_returns['eth_log_return'], 'ETH 对数收益率')
p_eth_return_lb = run_ljung_box_test(df_returns['eth_log_return'], 'ETH 对数收益率')

# --- 4. 一元时间序列模型 (ARIMA & GARCH) ---
print("\n--- 4. 一元时间序列模型 (ARIMA & 升级版 GARCH) ---")

if True:
    # --- 自动寻优的函数 (遍历 p,q) ---
    def auto_arima_select(ts, d=0, max_p=3, max_q=3):
        best_aic = np.inf
        best_bic = np.inf
        best_aic_res = None
        best_bic_res = None
        best_aic_order = None
        best_bic_order = None

        for p in range(max_p + 1):
            for q in range(max_q + 1):
                # 增加 (0,0,0) 的跳过, 因为 GARCH 会处理 const
                if p == 0 and q == 0:
                    continue
                try:
                    model = ARIMA(ts, order=(p, d, q))
                    res = model.fit()
                    if res.aic < best_aic:
                        best_aic = res.aic
                        best_aic_res = res
                        best_aic_order = (p, d, q)
                    if res.bic < best_bic:
                        best_bic = res.bic
                        best_bic_res = res
                        best_bic_order = (p, d, q)
                except Exception as e:
                    continue

        return {
            'best_aic_order': best_aic_order,
            'best_bic_order': best_bic_order,
            'best_aic_result': best_aic_res,
            'best_bic_result': best_bic_res

```

```

    }

# --- 残差诊断与显著性检验函数 ---
def arima_diagnostics(result, series_name="Series", lb_lags=[10, 20]):
    print(f"\n--- {series_name} ARIMA 模型摘要 ---")
    print(result.summary())

    print(f"\n{series_name} 参数显著性 (p-values):")
    pvals = result.pvalues
    for name, pval in pvals.items():
        print(f"    {name:20s} p = {pval:.6f} -> {'显著' if pval < 0.05 else '不显著'}")

    print(f"\n{series_name} 残差 Ljung-Box 检验 (lags = {lb_lags}):")
    for lag in lb_lags:
        try:
            # 使用 statsmodels 0.13+ 的 acorr_ljungbox 格式
            lb = acorr_ljungbox(result.resid, lags=[lag], return_df=True)
            pvalue = lb['lb_pvalue'].iloc[0]
            print(f"    lag={lag} -> lb_stat={lb['lb_stat'].iloc[0]:.4f},
p={pvalue:.6f} -> {'通过(白噪声)' if pvalue>0.05 else '未通过(非白噪声)'}")
        except Exception as e:
            print(f"    lag={lag} 检验失败: {e}")

    # 残差绘图
    resid = result.resid
    fig, axes = plt.subplots(2, 2, figsize=(12, 8))
    axes[0, 0].plot(resid); axes[0, 0].set_title(f"{series_name} 残差时序")
    plot_acf(resid, lags=30, ax=axes[0, 1]); axes[0, 1].set_title(f"{series_name} 残差 ACF")
    plot_pacf(resid, lags=30, ax=axes[1, 0]); axes[1, 0].set_title(f"{series_name} 残差 PACF")
    sm.qqplot(resid, line='s', ax=axes[1, 1]); axes[1, 1].set_title(f"{series_name} 残差 Q-Q 图")
    plt.suptitle(f"{series_name} ARIMA 残差诊断")
    plt.tight_layout(rect=[0, 0.03, 1, 0.95])
    plt.show()

# ===== 4.1 BTC ARIMA =====
print("\n--- 4.1 BTC 对数收益率 ARIMA 自动寻优 (基于 AIC/BIC) ---")
# 确保 df_returns 存在 (在 Section 2 中创建)
btc_res = auto_arima_select(df_returns['btc_log_return'], d=0, max_p=3, max_q=3)
print(f"BTC 最优 AIC 阶数: {btc_res['best_aic_order']}")
print(f"BTC 最优 BIC 阶数: {btc_res['best_bic_order']}")
if btc_res['best_aic_result'] is not None:
    arima_result = btc_res['best_aic_result'] # 用于预测
    arima_diagnostics(arima_result, series_name="BTC 对数收益率")
else:
    print("警告: 未能找到可拟合的 BTC ARIMA 模型。")

# --- 4.3 GARCH(1,1)-t 模型 (BTC 对数收益率) ---
print("\n--- 4.3 GARCH(1,1) 模型 (BTC 对数收益率) ---")

try:
    garch_model_btc = arch_model(df_returns['btc_log_return'].dropna() * 100,
                                vol='Garch', p=1, o=0, q=1, dist='t')
    garch_result_btc = garch_model_btc.fit(update_freq=5, disp='off')
    print("\nBTC GARCH(1,1)-t 模型摘要:")

```



```

        print(garch_result_btc.summary())
        print("正在显示图 5: BTC GARCH(1,1)-t 标准化残差...")
        garch_std_resid_btc = garch_result_btc.resid /
garch_result_btc.conditional_volatility
        plt.figure(figsize=(10, 4))
        plt.plot(garch_std_resid_btc)
        plt.title('图 5: BTC GARCH(1,1)-t 标准化残差')
        plt.show()

    except Exception as e:
        print(f"GARCH(1,1) 拟合失败: {e}")

# --- 4.4 ARIMA-GARCH 联合模型 (样本内拟合与样本外预测) ---
print("\n--- 4.4 ARIMA-GARCH 联合模型 (样本内拟合与样本外预测) ---")

    if 'arima_result' in locals() and 'garch_result_btc' in locals() and 'df_prices'
in locals():
        try:
            last_price = df_prices['btc_close'].iloc[-1]
            last_date = df_prices.index[-1]
            forecast_horizon = 10
            nu = garch_result_btc.params['nu']
            t_critical = t.ppf(0.975, nu)

            # --- 样本内拟合 ---
            in_sample_mu = arima_result.fittedvalues
            prev_actual_prices = df_prices['btc_close'].shift(1)
            in_sample_fitted_prices = prev_actual_prices * np.exp(in_sample_mu)

            in_sample_vol = garch_result_btc.conditional_volatility / 100
            in_sample_upper = prev_actual_prices * np.exp(in_sample_mu + t_critical *
in_sample_vol)
            in_sample_lower = prev_actual_prices * np.exp(in_sample_mu - t_critical *
in_sample_vol)

            # --- 样本外预测 ---
            arima_forecast = arima_result.get_forecast(steps=forecast_horizon)
            pred_returns = arima_forecast.predicted_mean
            garch_forecast = garch_result_btc.forecast(horizon=forecast_horizon,
reindex=False)
            pred_volatility = np.sqrt(garch_forecast.variance.iloc[-1].values) / 100

            cum_pred_returns = np.cumsum(pred_returns)
            cum_pred_volatility = np.sqrt(np.cumsum(pred_volatility**2))

            pred_prices = last_price * np.exp(cum_pred_returns)
            upper_prices = last_price * np.exp(cum_pred_returns + t_critical *
cum_pred_volatility)
            lower_prices = last_price * np.exp(cum_pred_returns - t_critical *
cum_pred_volatility)

            future_dates = pd.date_range(start=last_date + pd.Timedelta(days=1),
periods=forecast_horizon, freq='D')
            price_forecast_df = pd.DataFrame({
                'Predicted_Price': pred_prices,
                'Lower_Price_95': lower_prices,
                'Upper_Price_95': upper_prices
            }, index=future_dates)

            print(f"\n未来 {forecast_horizon} 天预测结果:")
            print(price_forecast_df)

```

```

# --- 绘图 1: 全历史 ---
print(f"正在显示图 5.4: 全历史样本内拟合与样本外预测...")
plt.figure(figsize=(14, 7))
plt.plot(df_prices.index, df_prices['btc_close'], color='blue', alpha=0.5,
label='历史真实价格')
plt.plot(in_sample_fitted_prices.index, in_sample_fitted_prices,
color='green', linestyle='--', alpha=0.8, linewidth=1, label='样本内拟合价格')
plt.plot(future_dates, price_forecast_df['Predicted_Price'], color='red',
marker='o', markersize=4, label='未来预测价格')
plt.fill_between(future_dates, price_forecast_df['Lower_Price_95'],
price_forecast_df['Upper_Price_95'], color='red', alpha=0.15, label='未来 95% 置信区间')

plt.title(f'图 5.4: BTC 价格全历史样本内拟合与未来 {forecast_horizon} 天预测')
plt.xlabel('日期')
plt.ylabel('BTC 价格 (USD)')
plt.legend(loc='upper left')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# --- 绘图 2: 局部放大 (2025-09-01 至今) ---
print(f"正在显示图 5.5: 局部样本内拟合与样本外预测 (2025 年 9 月至今)...")
start_date_zoom = '2025-09-01'
df_zoom = df_prices[start_date_zoom:]
fitted_zoom = in_sample_fitted_prices[start_date_zoom:]
lower_zoom = in_sample_lower[start_date_zoom:]
upper_zoom = in_sample_upper[start_date_zoom:]

if not df_zoom.empty:
    plt.figure(figsize=(14, 7))
    plt.plot(df_zoom.index, df_zoom['btc_close'], color='blue', marker='.',
linestyle='-', linewidth=2, alpha=0.6, label='真实价格')
    plt.plot(fitted_zoom.index, fitted_zoom, color='green', linestyle='--',
linewidth=2.5, alpha=0.9, label='拟合价格')
    plt.fill_between(fitted_zoom.index, lower_zoom, upper_zoom,
color='green', alpha=0.1, label='历史 95% 置信区间')
    plt.plot(future_dates, price_forecast_df['Predicted_Price'],
color='red', marker='o', markersize=6, linewidth=2, label='未来预测')
    plt.fill_between(future_dates, price_forecast_df['Lower_Price_95'],
price_forecast_df['Upper_Price_95'], color='red', alpha=0.25, label='未来 95% 置信区间')

    plt.title(f'图 5.5: 局部样本内拟合 (带历史置信区间) 与未来预测')
    plt.xlabel('日期')
    plt.ylabel('BTC 价格 (USD)')
    plt.legend(loc='upper left')
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.xticks(rotation=45)
    plt.show()
else:
    print(f"警告: 数据中没有 {start_date_zoom} 之后的数据。")

except Exception as e:
    print(f"模型拟合与预测失败: {e}")
else:
    print(f"错误: 缺少必要模型或数据。")

# --- 5. 多元时间序列分析 (VECM/VAR) ---

```

```

print("\n--- 5. 多元时间序列分析 (VECM / VAR) ---")

if 'df_prices' in locals() and 'df_returns' in locals():

    # --- 5.1 确定最优滞后阶数 ---
    print("\n--- 5.1 确定最优滞后阶数 (Lag Selection) ---")

    # A. VAR(df_returns) 滞后阶数
    model_returns = VAR(df_returns)
    lag_selection_returns = model_returns.select_order(maxlags=10)
    optimal_lag_returns = lag_selection_returns.aic
    print(f"VAR(df_returns) 最优滞后 (AIC): {optimal_lag_returns}")

    # B. VAR(df_prices) 滞后阶数
    model_prices = VAR(df_prices)
    lag_selection_prices = model_prices.select_order(maxlags=10)
    optimal_lag_prices = lag_selection_prices.aic
    print(f"VAR(df_prices) 最优滞后 (AIC): {optimal_lag_prices}")

    # --- 5.2 协整检验 (Johansen Test) ---
    print("\n--- 5.2 协整检验 (Johansen Test) ---")

    k_ar_diff = optimal_lag_prices - 1
    if k_ar_diff < 0: k_ar_diff = 0

    print(f"检验 'df_prices' (k_ar_diff = {k_ar_diff})")

    johansen_result = coint_johansen(df_prices, det_order=1, k_ar_diff=k_ar_diff)

    print("\n迹统计量 (Trace Statistic):")
    # H0: r=0
    print(f"    H0: r=0 (Trace Stat: {johansen_result.lr1[0]:.4f}, 95% Crit:
{johansen_result.cvt[0, 1]:.4f})")
    # H0: r<=1
    print(f"    H0: r<=1 (Trace Stat: {johansen_result.lr1[1]:.4f}, 95% Crit:
{johansen_result.cvt[1, 1]:.4f})")

    # --- 5.3 格兰杰因果检验 (Granger Causality Test) ---
    print("\n--- 5.3 格兰杰因果检验 (Granger Causality Test) ---")
    print(f"检验 'df_returns' (maxlag = {optimal_lag_returns})")

    # 检验 1: ETH 是否是 BTC 的格兰杰原因
    gc_result_eth_to_btc = grangercausalitytests(
        df_returns[['btc_log_return', 'eth_log_return']],
        maxlag=optimal_lag_returns,
        verbose=False
    )
    f_test_p_value_1 = gc_result_eth_to_btc[optimal_lag_returns][0]['ssr_ftest'][1]
    print(f"\nH0: ETH -/-> BTC (p-value): {f_test_p_value_1:.6f}")

    # 检验 2: BTC 是否是 ETH 的格兰杰原因
    gc_result_btc_to_eth = grangercausalitytests(
        df_returns[['eth_log_return', 'btc_log_return']],
        maxlag=optimal_lag_returns,
        verbose=False
    )
    f_test_p_value_2 = gc_result_btc_to_eth[optimal_lag_returns][0]['ssr_ftest'][1]
    print(f"\nH0: BTC -/-> ETH (p-value): {f_test_p_value_2:.6f}")

    # --- 5.4 拟合 VAR 模型 ---

```

```

print("\n--- 5.4 拟合 VAR 模型 (在收益率上) ---")

if 'df_returns' in locals() and 'optimal_lag_returns' in locals():
    try:
        # 1. 创建 VAR 模型实例
        model_var = VAR(df_returns)

        # 2. 拟合模型, 使用之前确定的最优滞后阶数 (10)
        print(f"正在拟合 VAR({optimal_lag_returns}) 模型...")
        var_result = model_var.fit(optimal_lag_returns)

        # 3. 输出模型摘要
        print("\nVAR 模型摘要:")
        print(var_result.summary())

        # 4. 检查模型的稳定性 (根是否在单位圆内)
        print("\nVAR 模型稳定性检查 (所有根的模应 < 1):")
        is_stable = var_result.is_stable()
        print(f"模型是否稳定: {is_stable}")

    except Exception as e:
        print(f"VAR 模型拟合失败: {e}")
else:
    print("错误: 缺少必要变量, 无法拟合 VAR 模型。")

# --- 5.5 脉冲响应分析 (IRF) ---
print("\n--- 5.5 脉冲响应分析 (Impulse Response Analysis) ---")

if 'var_result' in locals():
    try:
        # 设定分析的期数 (例如未来 20 天)
        irf_periods = 20

        irf = var_result.irf(irf_periods)

        print(f"正在显示图 6: 正交化脉冲响应函数 (未来 {irf_periods} 期)...")
        plt.figure(figsize=(12, 8))
        # orth=True 表示使用正交化冲击
        irf.plot(orth=True, impulse='btc_log_return',
response='eth_log_return', signif=0.05, subplot_params={'title': 'BTC 冲击 -> ETH 响应'})

        plt.show()

        plt.figure(figsize=(12, 8))
        irf.plot(orth=True, impulse='eth_log_return',
response='btc_log_return', signif=0.05, subplot_params={'title': 'ETH 冲击 -> BTC 响应'})

        plt.show()

        irf.plot(orth=True, signif=0.05, figsize=(12, 10))
        plt.suptitle('图 6: VAR(10) 正交化脉冲响应分析 (95% 置信区间)')
        plt.tight_layout(rect=[0, 0.03, 1, 0.95])
        plt.show()

    except Exception as e:
        print(f"脉冲响应分析失败: {e}")
else:
    print("错误: 缺少 VAR 模型结果 (var_result), 无法进行 IRF 分析。")

```

```

# --- 5.6 方差分解分析 (FEVD) ---
print("\n--- 5.6 方差分解分析 (Variance Decomposition) ---")

if 'var_result' in locals():
    try:
        # 设定分析期数 (例如未来 10 天)
        fevd_periods = 10

        # 计算方差分解
        fevd = var_result.fevd(fevd_periods)
        print(f"\n 方差分解结果 (未来 {fevd_periods} 天):")
        print(fevd.summary())
        print(f"正在显示图 7: 方差分解堆叠图...")
        # 绘制方差分解图
        fevd.plot(figsize=(12, 8))
        plt.suptitle(f'图 7: VAR(10) 方差分解 (未来 {fevd_periods} 天)')
        plt.tight_layout(rect=[0, 0.03, 1, 0.95])
        plt.show()

    except Exception as e:
        print(f"方差分解分析失败: {e}")
else:
    print("错误: 缺少 VAR 模型结果 (var_result), 无法进行 FEVD 分析。")

else:
    print("错误: 缺少 'df_prices' 或 'df_returns' DataFrame, 跳过多维分析。")

# --- 6. 风险价值 (VaR) 计算与回测 ---
print("\n--- 6. 风险价值 (VaR) 计算与回测 (基于 GARCH-t 模型) ---")

if 'garch_result_btc' in locals():
    try:
        # 1. 获取模型参数
        garch_params = garch_result_btc.params
        mu = garch_params['mu']
        nu = garch_params['nu'] # t 分布自由度

        # 2. 获取条件波动率 (Conditional Volatility)
        conditional_volatility = garch_result_btc.conditional_volatility
        # 3. 计算 95% VaR (alpha = 0.05)
        alpha = 0.05
        # t.ppf 是 t 分布的百分位点函数 (逆累积分布函数)
        q_t = t.ppf(alpha, nu)
        var_95 = (mu + conditional_volatility * q_t) / 100
        # 4. 创建用于绘图的 DataFrame
        var_df = pd.DataFrame({
            'Actual_Return': df_returns['btc_log_return'],
            'VaR_95': var_95
        }, index=df_returns.index).dropna()
        # 5. 绘制 VaR 回测图
        print("正在显示图 8: BTC 每日收益率与 95% VaR 对比图...")
        plt.figure(figsize=(12, 6))
        plt.plot(var_df.index, var_df['Actual_Return'], color='blue', alpha=0.5,
label='实际收益率')
        plt.plot(var_df.index, var_df['VaR_95'], color='red', linewidth=2,
label='95% VaR (GARCH-t)')
        plt.title('图 8: BTC 每日收益率 vs. 95% 动态风险价值 (VaR)')
        plt.legend()
        plt.grid(True, alpha=0.3)
    
```

```

plt.show()

# 6. 计算 VaR 突破次数 (Backtesting)
exceptions = var_df[var_df['Actual_Return'] < var_df['VaR_95']]
num_exceptions = len(exceptions)
total_obs = len(var_df)
actual_alpha = num_exceptions / total_obs

print(f"\nVaR 回测结果 (95% 置信水平, 理论突破率应接近 5%):")
print(f" 总观测天数: {total_obs}")
print(f" 实际突破次数: {num_exceptions}")
print(f" 实际突破率: {actual_alpha:.2%}")
if abs(actual_alpha - 0.05) < 0.01: # 简单判断, 实际应做 Kupiec 检验
    print(" -> 模型表现良好 (实际突破率接近理论值 5%)")
else:
    print(" -> 模型可能需要改进 (实际突破率偏离理论值)")

# 7. 预测下一交易日的 VaR
forecast = garch_result_btc.forecast(horizon=1)
next_vol = np.sqrt(forecast.variance.iloc[-1, 0])
next_var_95 = (mu + next_vol * q_t) / 100

print(f"\n未来一日风险预测:")
print(f" 下一交易日 95% VaR: {next_var_95:.4%}")
print(f" (解读: 有 95% 的把握, 明天 BTC 的损失不会超过 {abs(next_var_95):.2%})")

except Exception as e:
    print(f"VaR 计算失败: {e}")
else:
    print("错误: 缺少 GARCH 模型结果 (garch_result_btc), 无法计算 VaR。")

```